

人工智能复习

第一章——人工智能

关于智能的三大观点

思维理论

智能的核心是思维

知识阈值理论

强调知识对于智能的重要性

进化理论

智能可以通过逐步进化实现

第二章——知识工程

基本概念

知识

将有关信息关联在一起所形成的信息结构

知识表示方法

一阶谓词逻辑

产生式表示法

谓词

谓词的表示形式

• 谓词的一般形式

$$P(x_1, x_2, \dots, x_n)$$

其中， P 是谓词名， $x_1, x_2, \dots, x_n$ 是个体。谓词名通常用大写的英文

字母表示，个体通常用小写的英文字母表示。

例如：

➤ 命题的谓词表示：Friend (A, Brother (B)) :
A与B的哥哥是朋友



谓词公式的指派

□ 定义2.3 设 D 为谓词公式 P 的个体域，对 P 中个体常量、函数和谓词按如下规定赋值：

- (1) 为每个个体常量指派 D 中一个元素；
- (2) 为每个 n 元函数指派一个 $D^n \rightarrow D$ 的映射，

$$D^n = \{(x_1, \dots, x_n) | x_1, \dots, x_n \in D\};$$

- (3) 为每个 n 元谓词指派一个 $D^n \rightarrow \{F, T\}$ 的映射。

则称这些指派为公式 P （ n 元谓词）在 D 上的一个解释 I 。

对于谓词公式的真值，需要运行谓词公式的值指派和真值的指派

□ 例2.2 $D = \{1,2\}, B = (\forall x)(P(x) \rightarrow Q(f(x), b))$

解：令 $A = P(x) \rightarrow Q(f(x), b)$ ，给出如下解释：

- (1) 对个体常量 b 和函数 $f(x)$ 的值指派为：

b	$f(1)$	$f(2)$
1	1	2

个体常量指派 D 中一个元素
 n 元函数指派一个 $D^n \rightarrow D$

- (2) 对谓词的真值指派为：

n 元谓词指派一个 $D^n \rightarrow \{F, T\}$

$P(1)$	$P(2)$	$Q(1,1)$	$Q(2,1)$	$Q(1,2)$	$Q(2,2)$
T	F	T	F	$\times$	$\times$

指派指直接让某个变元获得一个具体值，通过一组指派获得一个谓词公式到具体个体上的解释

□ 谓词公式: $B = (\forall x)(P(x) \rightarrow Q(f(x), b))$, $D = \{1, 2\}$

□ 一个解释:

b	$P(1)$	$P(2)$	$Q(1,1)$	$Q(2,1)$
1	T	F	T	F

□ 在此解释下:

当 $x = 1$ 时, $P(1) = T$, $Q(1,1) = T$, 公式 A 真值为 T ;

当 $x = 2$ 时, $P(2) = F$, 公式 A 真值为 T ;

所以在此解释下, $B = (\forall x)A = T$ 。

谓词公式的特性

永真:

一个谓词公式的所有解释都是真值

(即一个谓词公式的所有具体取值都是真值 T)

相容性/可满足性

若一个谓词公式存在一个解释是真值, 为可满足

永假

一个谓词公式的所有解释都是假

等价式的证明

使用真值表法

□ 证明: $\neg(P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$

□ 真值表法

P	Q	$\neg(P \vee Q)$	$\neg P \wedge \neg Q$	$\neg(P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$
T	T	F	F	T
T	F	F	F	T
F	T	F	F	T
F	F	T	T	T

□ 证明拒取式: $\neg Q, P \rightarrow Q \Rightarrow \neg P$

➤ $A \Rightarrow B$ 当且仅当 $A \rightarrow B$ 为真

➤ 方法一: 真值表法

P	Q	$\neg P$	$\neg Q$	$P \rightarrow Q$	$\neg Q \wedge (P \rightarrow Q)$	$\neg Q \wedge (P \rightarrow Q) \rightarrow \neg P$
0	0	1	1	1	1	1
0	1	1	0	1	0	1
1	0	0	1	0	0	1
1	1	0	0	1	0	1

基于谓词公式的知识表示 ==重点==

事实/证据的表示

使用析取 $\vee$ 和合取 $\wedge$ 表示

事实: 用谓词公式的或/与形表示, 例如:

$$A \vee B \vee C, A \wedge B \wedge C$$

规则的表示

使用蕴含式表示: 如果X, 那么Y

$$X \rightarrow Y$$

基于谓词公式的知识表示实例:

- 周杰伦比他父亲有名
- 高扬是计算机系的学生，但他不喜欢编程
- 人人爱劳动

□ 定义如下谓词：

- Famous(x,y):x比y有名
- Computer(x):x是计算机系的学生
- Like(x,y):x喜欢y Love(x,y):x爱y Man(x):x是人

□ 然后用谓词公式表示

- Famous(Jay,father(Jay))
- Computer(gaoyang) $\wedge$ $\neg$ Like(gaoyang, programming)
- $\forall x(\text{Man}(x) \rightarrow \text{Love}(x, \text{labour}))$

- 首先定义一个谓词名：例如上述的Computer，个体x，然后利用两个谓词之间的关系组合而成谓词公式
- 组成方式：由“谓词 + 连接词 + 谓词”构成

产生式

事实的表示——使用n元组

三元组：老王年龄已40：(wang,age,40)

老王与老张是朋友：(friendship,wang,zhang)

四元组：表示不确定性的知识，增加一个置信度

(friendship,wang,zhang,0.8)

规则的表达

2. 规则的表达

基本形式： $P \rightarrow Q$ ，或者：*If P Then Q。*

产生式与谓词逻辑的区别 ==重点==

- 谓词逻辑只能表示精确的知识，且谓词逻辑的前提条件的匹配也必须是精确的
- 产生式可以表示不精确的知识，前提条件的匹配也可以不精确

产生式系统 ==重点==

一个产生式系统由三部分组成：

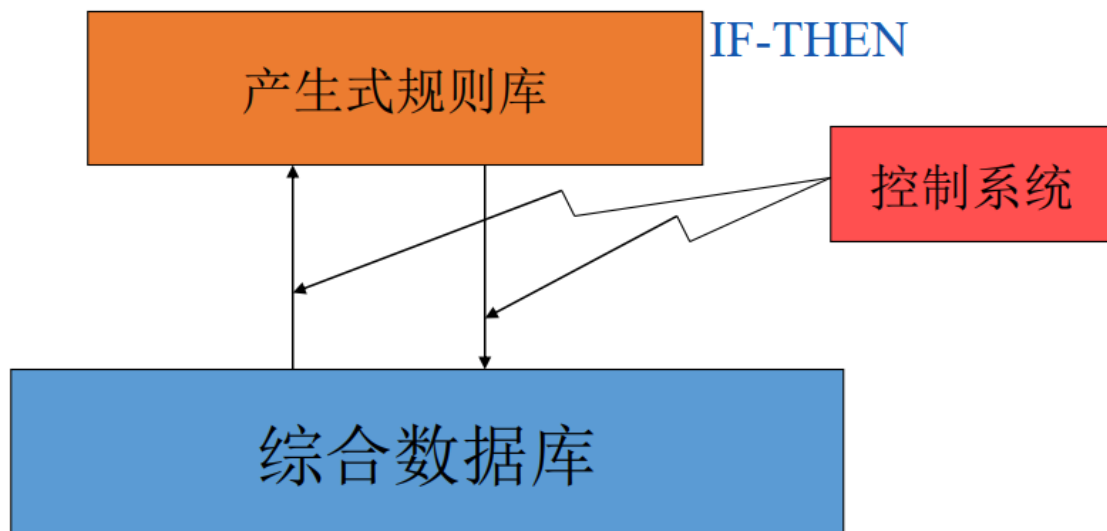
- **产生式规则库**（知识的产生式集合（即一群 $P \rightarrow Q$ 的知识集合））

产生式规则库是核心，存放着一群“IF P THEN Q”的以产生式表达形式存在的知识

- **综合数据库**（存放已知的事实和推导出的事实）
- **控制系统**

控制系统完成以下工作：

- 完成事实与规则的**匹配**，选出适用的规则集合
- 从选出的规则集合中**按照冲突消解策略**选取合适的一条规则用于推理
- 将推理后的结论放入数据库中
- 计算结论不确定性
- 决定何时停止运行



第三章——确定性推理

基本概念

推理

由已知判断推出另一个判断的思维过程

推理方式

演绎推理

从一般到特殊（从大到小），从一般性知识逐步推理到针对个别事物的新判断

演绎推理：从一般到特殊。例如三段论。

- 大前提（一般性知识）：运动员都很强壮
- 小前提（个别性知识）：小明是运动员
- 结论（针对个别事物的新判断）：小明很强壮

归纳推理

从个体到一般（从小到大），从足够多的个体实例归纳出一般性结论

归纳推理：从个体到一般。从足够多的个别实例归纳出一般性结论。

- 完全归纳推理：检查全部产品合格→该厂产品合格
- 不完全归纳推理：检查全部样品合格→该厂产品合格

推理分类

确定性推理：

所有知识都是**精确的**，按照因果关系进行逻辑推理

不确定性推理：

从不确定性证据出发，运用不确定性知识，推理出不确定性的结论

单调推理——演绎推理——推理越来越强

随着推理的进行，知识的不断加入，推论会不断加强，推理的结果越来越接近目标

非单调推理——默认推理——推论不能单调加强

随着推理的进行，知识的不断加入不仅不能加强推论，可能还要使推理退去

启发性推理

在推理过程中使用**与问题相关的，可以加快推理过程**的启发性知识的推理

非启发式推理

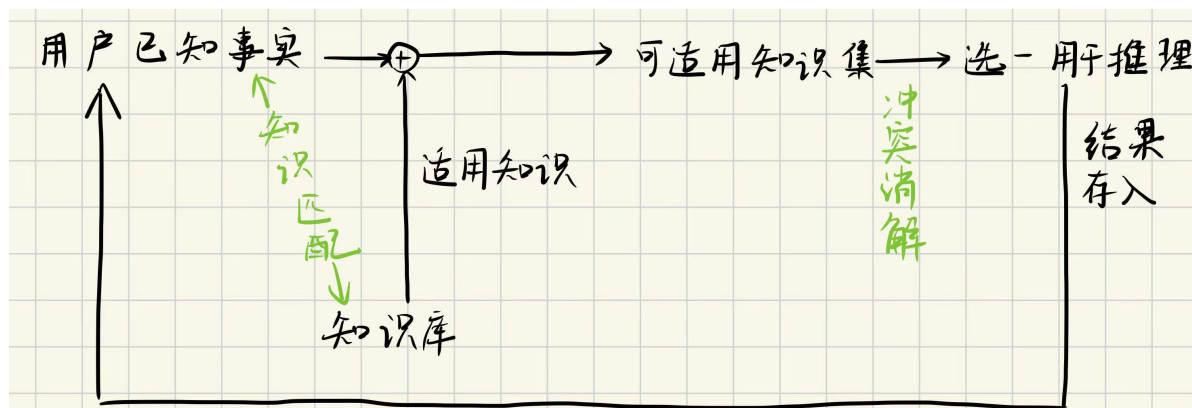
在推理过程中使用固定的一般的组合逻辑进行推理，可能产生“组合爆炸”

推理的控制策略

推理方向

正向推理——数据驱动推理

1. 从已知事实开始
2. 在知识库中找出适用——（搜索策略）的知识，构成适用的知识集合。*注：知识代表着事实到推理结果之间的映射关系，一条知识代表着一种证据到结论的映射
3. 按照冲突消解策略从适用的知识集合KS中选出一条知识进行推理，然后将推出的新事实重新加入数据库中作为下一步推理的已知事实
4. 直到推理得到想要的解



特点：容易实现，目的性不强但是效率低

逆向推理——目标驱动推理

1. 首选选取一个假设目标
2. 然后寻找支持该假设的证据
 - 若所需的证据都能找到，则原假设成立，推理完成
 - 若所需证据不一定全找到，则原假设不成立

特点：目的性强，但是初始假设不好选择，具有盲目性

搜索策略

搜索策略

- 从知识库寻找可用规则的策略。
- 状态空间搜索：宽度优先搜索、深度优先搜索、有界深度优先搜索。
- 启发式搜索。

冲突消解策略

冲突解决策略

- 事实与知识库中规则匹配的三种情况：恰好匹配成功（一对一）、不能匹配成功、多种匹配成功（一对多，多对一），多种匹配成功需要进行冲突消解。
- 常用冲突解决策略：按针对性排序、按已知事实的新鲜性排序、按匹配度排序、按条件个数排序。

通过**模式匹配**判断选中的知识集中哪一条知识能更完美的匹配当前的事实

求解策略

求几个目标解

限制策略

限制搜索的深度，宽度，时间等

知识的模式匹配

由于事实本身也是由知识模式组建而成的，在正向推理的选取合适的知识进行推理的模式匹配过程中需要对事实和知识库中的知识执行模糊匹配

但是知识库中的知识通常是**抽象**的，需要通过**变量代换**等形式让知识和事实的知识模板趋同，才能进行匹配

变量代换 == 重点 ==

□ 定义3-1 代换是一个形如

$$\{t_1/x_1, t_2/x_2, \dots, t_n/x_n\}$$

的有限集合。其中 $t_1, t_2, \dots, t_n$ 是项（常量、变量、函数）；

$x_1, x_2, \dots, x_n$ 是（某一公式中）互不相同的变元；

t_i/x_i 表示用 t_i 代换 x_i 。

规则：在代换中不允许出现代换的变量 t_i =被代换的变量 x_i ，也不允许被代换的变量 x_i 循环的出现在另一个代换变量 t_j 中

□ 不允许 t_i 与 x_i 相同，也不允许变元 x_i 循环地出现在另一个 t_j 中。

例如：

✓ $\{a/x, f(b)/y, w/z\}$ 是一个代换

✗ $\{g(y)/x, f(x)/y\}$ 不是代换→若改为 $\{g(a)/x, f(x)/y\}$ 即可

- 令 $\theta = \{t_1/x_1, t_2/x_2, \dots, t_n/x_n\}$ 为一个代换, F 为表达式, 则 $F\theta$ 表示对 F 用 t_i 代换 x_i 后得到的表达式。
- $F\theta$ 称为 F 的特例。

代换的复合==重点==

对于两个代换, 这两个代换的复合运算定义为:

□ 定义3-2 设

$$\theta = \{t_1/x_1, t_2/x_2, \dots, t_n/x_n\}$$

$$\lambda = \{u_1/y_1, u_2/y_2, \dots, u_m/y_m\}$$

是两个代换, 则这两个代换的复合也是一个代换, 它是从

$$\{t_1\lambda/x_1, t_2\lambda/x_2, \dots, t_n\lambda/x_n, u_1/y_1, u_2/y_2, \dots, u_m/y_m\}$$

中删去如下两种元素:

- ① $t_i\lambda/x_i$, 当 $t_i\lambda = x_i$ (项与变元相同)
- ② u_i/y_i , 当 $y_i \in \{x_1, x_2, \dots, x_n\}$ (变元已经被替换)

后剩下的元素所构成的集合, 记为 $\theta^\circ\lambda$ 。

代换的复合：

对于两个代换 $\theta = \{ t_1/x_1, t_2/x_2, t_3/x_3 \}$

$\lambda = \{ u_1/y_1, u_2/y_2, u_3/y_3, u_4/y_4 \}$

代换的复合 $\theta \circ \lambda =$ ① 首先对 θ 中的元素使用 λ 代换

$\Rightarrow \{ t_1\lambda/x_1, t_2\lambda/x_2, t_3\lambda/x_3 \}$

② 补充 λ 中的代换

$\Rightarrow \underbrace{\{ t_1\lambda/x_1, t_2\lambda/x_2, t_3\lambda/x_3 \}}_{\theta\lambda}, \underbrace{\{ u_1/y_1, u_2/y_2, u_3/y_3, u_4/y_4 \}}_{\text{原封不动地补充}\lambda}$

③ 删去不符合规则的项：

• 对于 $\theta\lambda$ ：删去 $t_i\lambda = x_i$ 的项

• 对于 λ ：删去已被替换过的变元

$y_i \in \{ x_1, x_2, x_3 \}$

~~是~~ 是 θ 中出现的元素，只要 θ 中出现过就删掉 ~~是~~

eg:

$\theta = \{ f(y)/x, z/y \}$

$\lambda = \{ a/x, b/y, y/z \}$

$\Rightarrow \theta \circ \lambda =$ ① $\{ f(y)\lambda/x, z\lambda/y \}$

② $\{ f(b)/x, \underline{y/y}, \underline{a/x}, b/y, y/z \}$

③ $\{ f(b)/x, y/z \}$

x, y 存在于 θ 中，删去

公式集的合一 == 重点 ==

如果两个知识（由谓词公式表示的公式集）经过代换后得到的谓词公式一致，则称这个代换为公式集的合一

□ **定义3-3** 设有公式集 $F = \{F_1, F_2, \dots, F_n\}$ ，若存在一个代换 λ 使得

$$F_1\lambda = F_2\lambda = \dots = F_n\lambda$$

则称 λ 为公式集 F 的一个**合一**，且称 $F_1, F_2, \dots, F_n$ 是**可合一**的。

例如，设有公式集

$$F = \{P(x, y, f(y)), P(a, g(x), z)\}$$

则下式是它的一个合一：

$$\lambda = \{a/x, g(a)/y, f(g(a))/z\}$$

F 中两个表示式都转换为 $P(a, g(a), f(g(a)))$

□ 一个公式集的合一**一般不唯一**。

最一般合一

对于一个公式集 F 的所有的合一 θ 都存在一个代换 λ ，使得这些合一可以通过一个**最一般合一**经过代换得到

定义3-4 设 σ 是公式集 F 的一个合一，如果对 F 的任一个合一 θ

都存在一个代换 λ ，使得 $\theta = \sigma \circ \lambda$

则称 σ 是一个**最一般的合一**（Most General Unifier, MGU）。

特点：一个公式集的合一不是唯一的，但是最一般合一是**唯一**的

如何求两个公式的最一般合一==重点==

- 使用**差异集**的概念，差异集定义为：

□ **差异集：**两个公式中相同位置处**不同符号的集合**。

例如：

$$F_1: P(x, y, z),$$

$$F_2: P(x, f(a), h(b))$$

分别从 F_1 与 F_2 的第一个符号开始，逐个向右比较，

则 F_1 与 F_2 的差异集：

$$D_1 = \{y, f(a)\}, \quad D_2 = \{z, h(b)\}$$

- MGU算法求取最一般合一：

□ 求取最一般合一的算法 (MGU算法) :

1. 令 $k = 0, F_k = F, \sigma_k = \varepsilon$ 。 ε 是空代换。
2. 若 F_k 只含一个表达式, 则算法停止, σ_k 就是最一般合一。
3. 找出 F_k 的差异集 D_k 。
4. 若 D_k 中存在元素 x_k 和 t_k , 其中 x_k 是变元, t_k 是项, 且 x_k 不在 t_k 中出现, 则置:

$$F_{k+1} = F_k \{t_k/x_k\} \quad \text{对 } F_k \text{ 用 } t_k \text{ 代换 } x_k$$

$$\sigma_{k+1} = \sigma_k \circ \{t_k/x_k\} \quad \text{计算复合代换}$$

$$k = k + 1$$

然后转步骤2。若不存在这样的 x_k 和 t_k 则算法停止。

5. 算法终止, F 的最一般合一不存在。

使用 MGU 算法 求解最一般合一:

eg: 公式集 $F = \{P(a, x, f(g(y))), P(z, f(z), f(u))\}$

① 初始化, 设 $F_0 = F$, 最一般合一 $\sigma_0 = \varepsilon$

② F_0 中有两个公式集, 不是最一般合一

③ 检查 F_0 差异集中的首元素 $D_0 = \{a, z\}$, 设置代换 $\lambda = \{a/z\}$

④ 计算代换:

$$\begin{cases} F_1 = F_0 \lambda = \{P(a, x, f(g(y))), P(a, f(a), f(u))\} \\ \sigma_1 = \sigma_0 \lambda = \{a/z\} \end{cases}$$

⑤ 递归 检查 F_1 的差异集中第 1 个位置的 D_1

$D_1 = \{x, f(a)\}$, 设为代换 $\lambda = \{x/f(a)\}$

$$\begin{cases} F_2 = F_1 \lambda = \{P(a, x, f(g(y))), P(a, x, f(u))\} \\ \sigma_2 = \sigma_1 \lambda = \{a/z, x/f(a)\} = \{a/z, x/f(a)\} \end{cases}$$

自然演绎推理

从一组已知为真的事实出发，直接运用经典逻辑的推理规则推出结论的过程

包括假言推理，P规则，拒取式推理等

归结演绎推理

将定理证明的问题转化为证明前提 $P \rightarrow$ 结论 Q 永真

即 $P \rightarrow Q$ 永真

使用反证法，证明其否定永假即可

海伯伦理论

□ 在谓词逻辑中，把原子谓词公式及其否定统称为文字。如：

$$P(x), \neg P(x, f(x)), Q(x, g(x))$$

□ 定义3-5：任何文字的析取式称为子句。

例如： $P(x) \vee Q(x)$,

$$\neg P(x, f(x)) \vee Q(x, g(x))$$

□ 定义3-6：不包含任何文字的子句称为空子句（ $\perp$ 或NIL）。

- 由于空子句不含有任何文字，也就不能被任何解释所满足，因此空子句是永假的，不可满足的。

将谓词公式化为子句集的步骤==重点==

归结演绎推理:

一、将原子谓词公式称为文字 (单个谓词)

二、将文字的析取式(或)称为子句 (谓词公式可以有其它运算)

$\Rightarrow$ 子句 $C = P(x) \vee Q(x)$ (但析取式的谓词公式叫子句)

三、对于多个子句 $C_1, C_2, \dots, C_n$, 可通过以下步骤化为:

合取范式形式, 形成子句集 S

$\Rightarrow$ 合取范式: $C_1 \wedge C_2 \wedge \dots \wedge C_n$

$\Rightarrow$ 子句集 $S = \{C_1, C_2, \dots, C_n\}$

三*. 任何谓词公式 F 都可化为相应子句集

即: 任何谓词公式 F 通过

① 变换

将谓词公式变换为多个文字 $\vee$ 文字的谓词公式形式

② 合成

多个子句之间经过变换化为合取范式, 得到子句集

如何将谓词公式 F 化为子句集:

① 消条件

利用等价关系消去连接词 " $\rightarrow$ " 和 " $\leftrightarrow$ "

$$\begin{cases} P \rightarrow Q \Leftrightarrow \neg P \wedge Q \\ P \leftrightarrow Q \Leftrightarrow (\neg P \wedge \neg Q) \vee (P \wedge Q) \end{cases}$$

② 减否定

利用等价关系将 " $\neg$ " 移到紧靠谓词的位置上, 并使每个否定符号最多作用于一个谓词

$$\begin{cases} \text{双重否定} \\ \text{de Morgan 律} \\ \text{量词转换: } \neg(\exists x)P \Leftrightarrow \forall(x)\neg P, \neg(\forall x)P \Leftrightarrow (\exists x)\neg P \end{cases}$$

不存在 x 任意 x 都不 不是任意 x 存在 x 都 P

③ 量词变元一致

重命名变元, 让不同的量词约束有不同的名称

$$\Rightarrow (\forall x)((\forall y)P \vee (\forall y)Q)$$

y 重命名 z

④ 消存在

消去存在量词 $\begin{cases} \text{若在全称量词处 (即括号外先有存在, 再有 } \forall) \\ \text{替换为一新变元} \end{cases}$

⑤ 化前束

将剩余的量词 (仅剩全称量词) 全部移到公式左边

(可以直接移, 变元一致过程中让每个变元仅有一个约束)

⑥ 化 skolem 合取式

利用 **分配律** 将公式化为 **合取范式**

$$P \vee (Q \wedge R) \iff (P \vee Q) \wedge (P \vee R)$$

⑦ 消全称

消去全称量词，仅保留合取范式

⑧ 化子句

将合取式转化为子句集的形式

因此可以将归结演绎的核心转变为：

从证明 $P \rightarrow Q$ 永真 ($\sim P \vee Q$ 永真) 转变为证明 $(P \wedge \sim Q)$ 永假，即子句集 S 不可满足

命题逻辑中的归结原理——鲁滨逊归结原理 ==重点==

如何判断子句集 S 是否可满足呢，运用鲁滨逊归结原理：

- 若一个谓词公式的子句集 S 中包含**空子句**，则子句集 S 不可满足
- 若该子句集 S 中的子句 $C_1, C_2, \dots$ 通过归结能推出空子句，那么子句集 S 不可满足

归结 ==重点==

□ 定义3-9 若 P 是原子谓词公式，则称 P 与 $\neg P$ 为**互补文字**。

在命题逻辑中， P 为命题。

1. 对于两个子句 C_1 和 C_2 ，首先**消去互补的文字**
2. 将两个子句剩下的部分进行**析取 $\vee$** ，得到一个新的子句 C_{12} ，称为归结式

□ 例：设 $C_1 = \neg P \vee Q, C_2 = \neg Q \vee R, C_3 = P$ ，求 C_1, C_2, C_3 的归结式 C_{123} 。

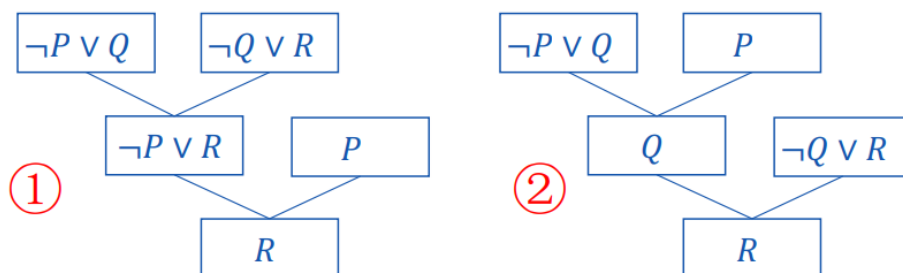
□ 解：①先对 C_1 和 C_2 归结，可得 $C_{12} = \neg P \vee R$

然后对 C_{12} 和 C_3 归结，可得 $C_{123} = R$

②若改变顺序，先对 C_1 和 C_3 归结，可得 $C_{13} = Q$

然后对 C_{13} 和 C_2 归结，亦可得 $C_{123} = R$

□ 归结过程不唯一。可用下图的归结树表示。



归结式是其亲本子句的逻辑结论

□ 推论1 设 C_1 与 C_2 是子句集 S 中的两个子句， C_{12} 是它们的归结式。若用 C_{12} 代替 C_1 和 C_2 后得到新子句集 S_1 ，则由 S_1 的不可满足性可推出原子句集 S 的不可满足性，即

S_1 的不可满足性 $\Rightarrow S$ 的不可满足性

$$S_1 = \{C_{12}, C_3\} \quad S = \{C_1, C_2, C_3\}$$

□ 推论2 设 C_1 与 C_2 是子句集 S 中的两个子句， C_{12} 是它们的归结式。若把 C_{12} 加入 S 中得到新子句集 S_2 ，则 S 与 S_2 在不可满足的意义上是等价的，即

S_2 的不可满足性 $\Leftrightarrow S$ 的不可满足性

$$S_2 = \{C_{12}, C_1, C_2, C_3\} \quad S = \{C_1, C_2, C_3\}$$

由于归结式的不可满足性可以替代原子句集的不可满足性，那么当归结式不可满足（归结式出现空子句），那么原子句集一定不可满足

若原子句集不可满足，一定能归结出空子句

代换后归结

归结的前提是存在互补文字，当子句中两个谓词公式的变元不同时，不存在互补文字，需要用最一般合一一对变元进行代换，得到两个相同变元的文字

二元归结式

□ **定义3-11** 设 C_1 与 C_2 是两个没有相同变元的子句， L_1 和 $\neg L_2$ 分别是 C_1 和 C_2 中的文字。若 σ 是 L_1 和 L_2 的最一般合一，则称

$$C_{12} = (C_1\sigma - \{L_1\sigma\}) \vee (C_2\sigma - \{\neg L_2\sigma\})$$

为 C_1 和 C_2 的二元归结式， L_1 和 $\neg L_2$ 称为归结式上的文字。

意思就是， L_1 和 $\neg L_2$ 是预备要在归结中删去的文字，将 C_1 和 C_2 经过 σ 合一代换后减去互补文字的剩下的部分析取 $\vee$ 在一起得到的归结式

归结反演

□ 如欲证明 Q 为 $P_1, P_2, \dots, P_n$ 的逻辑结论，
只需证明 $(P_1 \wedge P_2 \wedge \dots \wedge P_n) \rightarrow Q$ 永真，
即 $\neg (P_1 \wedge P_2 \wedge \dots \wedge P_n) \vee Q$ 永真，
或证明 $(P_1 \wedge P_2 \wedge \dots \wedge P_n) \wedge \neg Q$ 是不可满足的，
或证明 其子句集是不可满足的。

□ 而子句集的不可满足性可用归结原理来证明。

□ 应用归结原理证明定理的过程称为归结反演。

使用归结原理证明定理的过程称为归结反演

归结反演求解的步骤： ==重点==

对于公式集 F ，目标是 Q ，用归结反演证明目标 Q 为真

核心思路是：对于 $F \rightarrow Q$ ，求出其反演为永假

1. 给目标 Q 取反，得到 $\neg Q$
2. 将 $\neg Q$ 并入公式集中，得到 $\{F, \neg Q\}$
3. 将公式集化为子句集 S
4. 利用**归结原理**，对子句集 S 进行归结，若得到**空子句**即证明
5. 归结时，若变元不一致，先使用最一般合一进行代换

归结策略

对于一个子句集 $S=\{C_1, C_2, C_3, C_4\}$ ，如何合适的选择两个子句进行归结，使得归结效率更高

删除策略——删去无用的子句

尽早删去无用子句，避免无效归结

- **纯文字删除法**：子句中不存在互补文字的文字可直接删除
- **重言式删除法**：一个子句中同时包含**互补文字对**
- **包孕删除法**：若一个子句 C_1 可以由子句 C_2 代换产生，则子句 C_1 包孕于 C_2 ，可删除

□ 包孕删除法

- 设有子句 C_1 和 C_2 ，如果存在一个代换 σ ，使得 $C_1\sigma \subseteq C_2$ （子集），则称 C_1 **包孕**于 C_2 ， C_2 可删除。
- 例如： $C_1 = P(x)$ ， $C_2 = P(y) \vee Q(z)$ ，则 C_1 包孕于 C_2

限制策略——对参加归结的子句进行限制，减小归结盲目性

- 支持集策略：亲本子句**必须是通过目标子句的否定归结而来的**——完备
- 线性输入策略：参加归结的子句中**必须至少有一个是亲本子句集中的子句**——不完备
- 祖先过滤策略
- 单文字子句策略：参加归结的子句**必须至少有一个是单文字子句**——不完备

第四章——不确定性推理

基本概念

不确定性推理

不确定性推理就是从**不确定性的初始证据**出发，运用**不确定性的知识**推导出**不确定性结论**的过程

不确定性证据：

不确定性证据表示证据的不确定性程度，在可信度模型中代表证据的**可信度** $CF(E)$ ，这个值通常是动态的，称为动态强度

不确定性知识：

不确定性知识表示：**证据与结论之间的关联规则和关联程度**，在可信度模型中不确定性知识代表着**证据对结论的支持程度**，即 $CF(H,E)$ ，这个值通常是静态的，称为静态强度

不确定性的表示

LS与LN

LS是充分性度量——指出证据对结论的支持程度，LS越大说明证据对结论越有利

LN是必要性度量——指出证据对结论的反对程度，LN越大说明证据对结论越不利

一般情况下 $LS > 1$ 而 $LN < 1$

LS是充分性度量，指出 E 对 H 的支持程度，取值范围为 $[0, \infty)$

$$LS = \frac{P(E|H)}{P(E|\neg H)}$$

LN是必要性度量，指出 $\neg E$ 对 H 的支持程度，即 E 对 H 为真的必要程度，取值范围为 $[0, \infty)$

$$LN = \frac{P(\neg E|H)}{P(\neg E|\neg H)} = \frac{1 - P(E|H)}{1 - P(E|\neg H)}$$

LS 和 LN 的值由领域专家给出，代表知识的**静态强度**。

4.4可信度方法

基本概念

可信度

对一个事物和现象真的相信的程度

C-F模型

可信度使用CF模型表示，CF模型分为**静态强度**和**动态强度**

$$\text{IF } E \text{ THEN } H \text{ (CF(H,E))}$$

其中， **$CF(H,E)$** 是该知识的可信度，称为可信度因子或规则强度，表示 E 为真时对 H 为真的支持程度，即静态强度。

此处 $CF(H,E) \in [-1,1]$ 。

- 静态强度指的是：证据对结论的支持程度，越大证据越能支持结论—— $C(H,E)$

$CF(H,E) = 1$ 对应于 $P(H|E) = 1$ （证据绝对支持结论）

$CF(H,E) = -1$ 对应于 $P(H|E) = 0$ （证据绝对否定结论）

$CF(H,E) = 0$ 对应于 $P(H|E) = P(H)$ （证据与结论无关）

- 动态强度指的是：证据本身的可信度，越大说明该证据越可信—— $C(E)$

□ 证据的不确定性也用可信度因子表示。如： $CF(E) = 0.6$ 表示证据 E 的可信度为0.6。

$CF(E)$ 的取值范围： $[-1, +1]$ 。

$CF(E) > 0$ ：表示证据以某种程度为真。

$CF(E) < 0$ ：表示证据以某种程度为假。

$CF(E) = 0$ ：表示无法知道证据的真假。

带阈值限度的可信度模型

在带阈值限度的可信度模型中，知识的动态强度 $C(E)$ 和静态强度 $C(H, E)$ 的取值范围都是 $[0, 1]$ ，与原始的C-F模型不同

$CF(H, E)$ 为知识的可信度，取值范围为 $[0, 1]$ 。

$CF(H, E) = 0$ 对应于 $P(H|E) = 0$ （证据绝对否定结论）

$CF(H, E) = 1$ 对应于 $P(H|E) = 1$ （证据绝对支持结论）

证据 E 的可信度仍为 $CF(E)$ ，但其取值范围为： $[0, 1]$

- $CF(E) = 1$ 表示证据绝对存在；
- $CF(E) = 0$ 表示证据绝对不存在。

在带阈值限度的可信度模型中，只有当动态强度——知识的可信度大于等于阈值的时候，这个知识才能被应用

□ 知识用下述形式表示：

$$\text{IF } E \text{ THEN } H (CF(H, E), \lambda)$$

其中：

- $CF(H, E)$ 为知识的可信度，取值范围为 $[0, 1]$ 。

$CF(H, E) = 0$ 对应于 $P(H|E) = 0$ （证据绝对否定结论）

$CF(H, E) = 1$ 对应于 $P(H|E) = 1$ （证据绝对支持结论）

- λ 是阈值，明确规定了知识运用的条件：只有当 $CF(E) \geq \lambda$ 时，该知识才能够被应用。 λ 的取值范围为 $[0, 1]$ 。

组合证据的不确定性

若当前证据是组合证据，其不确定性定义为：

(1) 当组合证据是多个单一证据的合取时，即

$$E = E_1 \text{ AND } E_2 \text{ AND } \dots \text{ AND } E_n$$

则： $CF(E) = \min\{CF(E_1), CF(E_2), \dots, CF(E_n)\}$

(2) 当组合证据是多个单一证据的析取时，即

$$E = E_1 \text{ OR } E_2 \text{ OR } \dots \text{ OR } E_n$$

则： $CF(E) = \max\{CF(E_1), CF(E_2), \dots, CF(E_n)\}$

不确定性的传递算法

当证据的可信度大于阈值时，要计算出结论的可信度，可以使用：

$$\text{结论的可信度 } C(H) = \text{证据支持结论的程度 } C(H, E) * \text{证据的可信度 } C(E)$$

□ 当 $CF(E) \geq \lambda$ 时，结论 H 的可信度 $CF(H)$ 由下式计算得到：

$$CF(H) = CF(H, E) \times CF(E)$$

- 注意： $CF(H, E)$ 表示证据 E 为真的条件下结论 H 为真的可能性。

不确定性的合成算法

假设多条不确定性规则具有相同的结论，那么每个证据推导出的结论的可信度都不同，那么如何求得其综合可信度呢？

- 对于多条规则，首先求出其每条规则的结论的可信度

设有多条规则有相同的结论，即

$$\text{IF } E_1 \text{ THEN } H \quad (CF(H, E_1), \lambda_1)$$

$$\text{IF } E_2 \text{ THEN } H \quad (CF(H, E_2), \lambda_2)$$

...

$$\text{IF } E_n \text{ THEN } H \quad (CF(H, E_n), \lambda_n)$$

如果这 n 条规则都满足： $CF(E_i) \geq \lambda_i, i = 1, 2, \dots, n$ 且都被启用

- ① 首先分别对每条知识求出它对结论 H 的可信度 $CF_i(H)$ ；

$$CF_i(H) = CF(H, E_i) \times CF(E_i)$$

- 然后使用综合可信度的求解方法：

- 极大值法：

$$CF(H) = \max\{CF_1(H), CF_2(H), \dots, CF_n(H)\}$$

- 加权求和法： $CF(H, E_i)$ 作为权重

$$CF(H) = \frac{1}{\sum_{i=1}^n CF(H, E_i)} \sum_{i=1}^n CF(H, E_i) \times CF(E_i)$$

- 有限和法：各结论 H 的可信度和不能大于1，否则 $CF(H)$ 取1

$$CF(H) = \min\left\{\sum_{i=1}^n CF_i(H), 1\right\}$$

加权的可信度模型

对于一个复合的证据（由多个子证据组合而成），其中的每一个子证据的重要性不是完全平等的，此时需要引入每个子证据的权重来表示

知识的不确定性

IF $E_1(\omega_1)$ AND $E_2(\omega_2)$ AND... AND $E_n(\omega_n)$
THEN H ($CF(H, E), \lambda$)

其中 $\omega_i (i = 1, 2, \dots, n)$ 是加权因子, λ 是阈值, 其值均由专家给出。

加权因子的取值范围一般为 $[0, 1]$, 且应满足归一条件,

$$\text{即 } 0 \leq \omega_i \leq 1, \sum_{i=1}^n \omega_i = 1 \quad i = 1, 2, \dots, n$$

注意: 权重应当满足归一条件

组合证据的不确定性

在加权可信度模型中, 对于组合证据 $E_1, E_2, \dots, E_n$ 的可信度

组合证据的不确定性

若有 $CF(E_1), CF(E_2), \dots, CF(E_n)$, (证据 E_i 的可信度为 $CF(E_i)$), 则组合证据的可信度为:

$$CF(E) = \frac{1}{\sum_{i=1}^n \omega_i} \sum_{i=1}^n \omega_i \times CF(E_i) \quad \text{不满足归一条件}$$

$$CF(E) = \sum_{i=1}^n \omega_i \times CF(E_i) \quad \text{满足归一条件}$$

不确定性的传递算法、

与带阈值限度的可信度模型相同

不确定性的传递算法

当一条知识的 $CF(E)$ 满足如下条件时,

$$CF(E) \geq \lambda$$

该知识就可被应用。结论 H 的可信度为:

$$CF(H) = CF(H, E) \times CF(E)$$

举例:

□ 证据不完全

IF 该动物有蹄 (0.3) -- E_1
AND 该动物有长腿 (0.2) -- E_2
AND 该动物有长颈 (0.2) -- E_3
AND 该动物是黄褐色 (0.13) -- E_4
AND 该动物身上有暗黑色斑点
(0.13) -- E_5
AND 该动物的体重在200KG以上
(0.04) -- E_6
THEN 该动物是长颈鹿 (0.95, 0.8)
-- H

权重

证据可信度

- 小朋友在幼儿园看到一个动物:
该动物有蹄, 可信度为1
该动物有长腿, 可信度为1
该动物有长颈, 可信度为1
该动物是黄褐色, 可信度为0.8
该动物身上有暗黑色斑点, 可信度为0.6
- 那么该动物是什么动物?
- 很显然证据 E_6 缺失, 按照不带加权因子的推理模型无法得到答案

- 小朋友的观察 (组合证据):

$$CF(E) = \sum_{i=1}^n \omega_i \times CF(E_i)$$
$$= 0.3 \times 1 + 0.2 \times 1 + 0.2 \times 1 + 0.13 \times 0.8 + 0.13 \times 0.6 = 0.882$$

- 因为 $CF(E) \geq \lambda = 0.8$, 则该知识可以被应用, 那么可以推出结论 H (该动物是长颈鹿) 的可信度为:

$$CF(H) = CF(H, E) \times CF(E) = 0.95 \times 0.882 = 0.84$$

4.5 模糊推理

模糊理论的基本概念

模糊集

模糊集用来描述一个模糊概念，在一个论域中的每个元素隶属于这个模糊概念的程度称为隶属度

例如，模糊集A描述模糊概念“好”，论域中包括两个元素：{优秀, 一般}

其中“优秀”对于模糊概念“好”的隶属度为1

“一般”对于模糊概念“好”的隶属度为0.5,

那么模糊集“好”在这个论域上的模糊集就是{1/优秀, 0.5/一般}

1. 模糊集

□ 定义4.4 设 U 是论域， μ_A 是把任意 $u \in U$ 映射为 $[0, 1]$ 上某个值的函数，即

$$\mu_A: U \rightarrow [0, 1]$$

则：

μ_A 是定义在 U 上的一个隶属函数。

$A = \{\mu_A(u), u \in U\}$ 称为 U 上的一个模糊集。

$\mu_A(u)$ 称为 u 对模糊集 A 的隶属度。

μ_A 称为模糊集 A 的隶属函数。

- 模糊集与隶属函数则是一一对应的，一个模糊集必有一个独一无二的隶属函数与之对应，反之亦然。



隶属函数

每一个模糊集都有一个对应的隶属函数——对应，隶属函数描述了模糊集中每个元素隶属于当前模糊集合的隶属度，隶属度越高表示越属于当前模糊集

若论域离散且有限，则模糊集A可表示为：

$$A = \{\mu_A(u_1), \mu_A(u_2), \dots, \mu_A(u_n)\}$$

也可写为：

$$A = \mu_A(u_1)/u_1 + \mu_A(u_2)/u_2 + \dots + \mu_A(u_n)/u_n$$
$$A = \sum_{i=1}^n \mu_A(u_i)/u_i, \text{ 或者 } A = \bigcup_{i=1}^n \mu_A(u_i)/u_i$$

或者：

$$A = \{\mu_A(u_1)/u_1, \mu_A(u_2)/u_2, \dots, \mu_A(u_n)/u_n\}$$
$$A = \{(\mu_A(u_1), u_1), (\mu_A(u_2), u_2), \dots, (\mu_A(u_n), u_n)\}$$

- 隶属度为0的元素可以不写。
- $\mu_A(u_i)/u_i$ 表示论域中的元素与其隶属度之间的对应关系，并不表示“分数”；“+”表示模糊集在论域U上的整体，并不表示“求和”。

对于一个模糊集合A：

- 如果其隶属函数 $\mu(u)$ 的取值为0，则模糊集合A变为空集
- 若隶属函数取值为1，则模糊集合A是经典集合

在给定的论域U上可以定义多个模糊集。U上的全体模糊集记为：

$$F(U) = \{A | \mu_A: U \rightarrow [0,1]\}$$

或

$$F(U) = \{\mu_A | \mu_A: U \rightarrow [0,1]\}$$

模糊集的运算

包含

对于同一个论域U上的多个模糊集A,B，若对于论域中的所有元素而言，模糊集A的隶属度大于等于模糊集B的隶属度

$$\mu_B(u) \leq \mu_A(u)$$

则说明模糊集A包含B

记为 $B \subseteq A$ 。

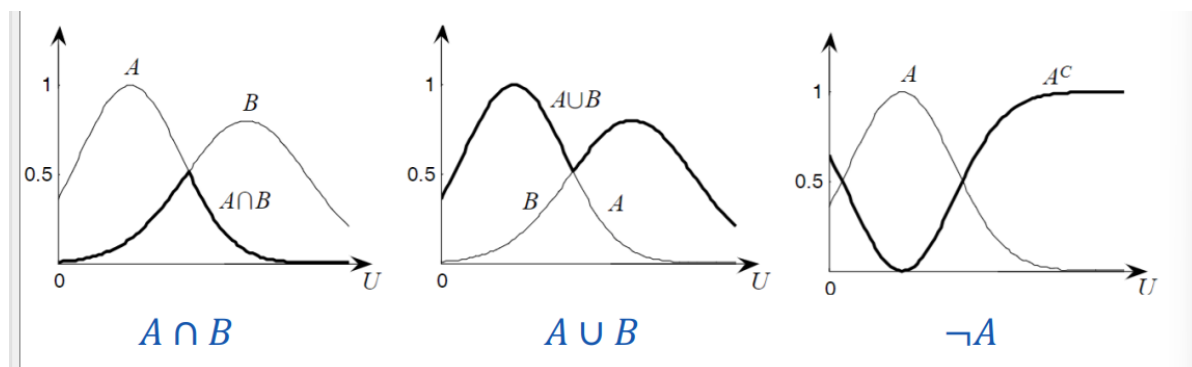
交、并、补

对于同一个论域上的模糊集合A,B, 其交集, 并集和补集的运算为:

$$A \cup B: \mu_{A \cup B}(u) = \max\{\mu_A(u), \mu_B(u)\} = \mu_A(u) \vee \mu_B(u)$$

$$A \cap B: \mu_{A \cap B}(u) = \min\{\mu_A(u), \mu_B(u)\} = \mu_A(u) \wedge \mu_B(u)$$

$$\neg A: \mu_{\neg A}(u) = 1 - \mu_A(u)$$



即:

- 交集时两个模糊集合上的对应元素取两个集合中最小的
- 并运算时两个模糊集上对应元素取两个模糊集中最大的元素
- 补运算时每个模糊集上每个元素的隶属度取反 (1-当前隶属度)

例如:

已知两个模糊集 A, B

$$A = \{0.9, 0.2, 0.8, 0.5\}$$

$$B = \{0.3, 0.1, 0.4, 0.6\}$$

$$\text{则 } A \cup B = \text{取大者} = \{0.9, 0.2, 0.8, 0.6\}$$

$$A \cap B = \text{取小者} = \{0.3, 0.1, 0.4, 0.5\}$$

模糊关系

笛卡尔积

给定集合 X 和 Y ，由全体 $(x, y) (x \in X, y \in Y)$ 组成的集合，叫做 X 和 Y 的**笛卡尔积**（或叫直积），记作 $X \times Y$

$$X \times Y = \{(x, y) | x \in X, y \in Y\}$$

$X \times Y$ 表示由两个集合中所有元素组合而成的二元关系的集合

模糊关系

模糊关系指的是**笛卡尔积**集合 $\{X \times Y\}$ 上的一个模糊集，即**模糊关系就是模糊集**

模糊集的笛卡尔积

对于两个论域 U_1 和 U_2 上的模糊集 A_1 和 A_2 ，模糊集 A_1 和 A_2 的笛卡尔积定义为：

$$A_1 \times A_2 \times \cdots \times A_n = \int_{U_1 \times U_2 \times \cdots \times U_n} \frac{\mu_{A_1}(u_1) \wedge \mu_{A_2}(u_2) \wedge \cdots \wedge \mu_{A_n}(u_n)}{u_1, u_2, \dots, u_n}$$

为 $A_1, A_2, \dots, A_n$ 的笛卡儿乘积，它是 $U_1 \times U_2 \times \cdots \times U_n$ 上的一个模糊集。

A_1 和 A_2 的笛卡尔积定义为，每个对应位置的元素**取最小**

如“体育好”，则 A 与 B 的笛卡尔积 $A \times B$ 是 $U \times V$ 上的模糊集（即一种特殊的模糊关系），记为：

$$A \times B = \int_{U \times V} \frac{\mu_A(u) \wedge \mu_B(v)}{(u, v)}$$

其中“ $\wedge$ ”表示取最小值， $\mu_A(u)$ 是 u 对 A 的隶属度， $\mu_B(v)$ 是 v 对 B 的隶属度。

论域 $U = \{u_1, u_2\}$ 表示学习成绩，其中 u_1 可能代表“优秀”， u_2 代表“一般”。

论域 $V = \{v_1, v_2\}$ 表示体育成绩，其中 v_1 可能代表“优秀”， v_2 代表“一般”。

模糊集 $A_1 = \frac{1}{u_1} + \frac{0.5}{u_2}$ 表示“学习成绩好”的程度。

模糊集 $A_2 = \frac{0.8}{v_1} + \frac{0.6}{v_2}$ 表示“体育成绩好”的程度。

A_1 和 A_2 的笛卡尔积 $A_1 \times A_2$ 是定义在 $U \times V$ 上的模糊集，表示“学习成绩好且体育成绩好”的联合条件。

$$A_1 \times A_2 = \frac{0.8}{(u_1, v_1)} + \frac{0.6}{(u_1, v_2)} + \frac{0.5}{(u_2, v_1)} + \frac{0.5}{(u_2, v_2)}$$

定义另一个模糊关系 R 在 $U \times V$ 上，表示“学习成绩和体育成绩平衡”的程度。

$$R = \frac{0.9}{(u_1, v_1)} + \frac{0.3}{(u_1, v_2)} + \frac{0.2}{(u_2, v_1)} + \frac{0.6}{(u_2, v_2)}$$

模糊集的笛卡尔积是一种特殊的模糊关系。

计算笛卡尔积上的模糊关系R时，本质是计算一个论域是笛卡尔积的模糊集，只需要使用一个隶属函数进行计算即可得到模糊关系R，表示为：

- 当U和V都是有限论域时，其模糊关系R可用一个矩阵表示。

$$U = \{u_1, u_2, \dots, u_m\}$$

$$V = \{v_1, v_2, \dots, v_n\}$$

则 $U \times V$ 上的模糊关系可以表示为

$$R = \begin{bmatrix} \mu_R(u_1, v_1) & \mu_R(u_1, v_2) & \cdots & \mu_R(u_1, v_n) \\ \mu_R(u_2, v_1) & \mu_R(u_2, v_2) & \cdots & \mu_R(u_2, v_n) \\ \vdots & \vdots & \ddots & \vdots \\ \mu_R(u_m, v_1) & \mu_R(u_m, v_2) & \cdots & \mu_R(u_m, v_n) \end{bmatrix}$$

- $U \times V = \{(u, v) | u \in U, v \in V\}$ 中的一个模糊关系R是指以 $U \times V$ 为论域的一个模糊子集， (u, v) 的隶属度为 $\mu_R(u, v)$ 。

例 设 $U = V = \{u_1, u_2, u_3\}$ ，R是“信任关系”，则可有

$$R = \begin{bmatrix} 1 & 0.3 & 0.8 \\ 0.9 & 1 & 0.6 \\ 0.7 & 0.5 & 1 \end{bmatrix}$$

模糊关系的合成

对于隶属于两个笛卡尔积的两个模糊关系R1和R2，其合成运算规则为：

类似于矩阵乘积，不过乘法替换为“取小”，加法替换为“取大”

例 设有两个模糊关系

$$R_1 = \begin{bmatrix} 0.4 & 0.5 & 0.1 \\ 0.2 & 0.6 & 0.2 \\ 0.5 & 0.3 & 0.2 \end{bmatrix} \quad R_2 = \begin{bmatrix} 0.2 & 0.8 \\ 0.4 & 0.6 \\ 0.6 & 0.4 \end{bmatrix}$$

则 R_1 与 R_2 的合成是

$$R = R_1 \circ R_2 = \begin{bmatrix} 0.4 & 0.5 \\ 0.4 & 0.6 \\ 0.3 & 0.5 \end{bmatrix}$$

合成法则类似与矩阵乘法。

匹配度

如何评判两个模糊关系之间的相似度呢？——引入匹配度问题

匹配度可以用**语义距离** $d(A,B)$ 表示：

- 匹配度表示为：

$$1 - d(A, B)$$

- 海明距离：两个模糊关系的对应的元素的隶属度之间的距离：

海明距离

$$d(A, B) = \frac{1}{n} \times \sum_{i=1}^n |\mu_A(u_i) - \mu_B(u_i)|$$
$$d(A, B) = \frac{1}{b-a} \int_a^b |\mu_A(u) - \mu_B(u)| du$$

即两个模糊关系之间的距离越短，两个模糊关系的匹配度越高

设 $U = \{a, b, c, d\}$

$$A = 0.3/a + 0.4/b + 0.6/c + 0.8/d$$

$$B = 0.2/a + 0.5/b + 0.6/c + 0.7/d$$

语义距离（海明距离）：

$$d(A, B) = \frac{1}{4} \times (|0.3 - 0.2| + |0.4 - 0.5| + |0.6 - 0.6| + |0.8 - 0.7|) = 0.075$$

$$(A, B) = 1 - d(A, B) = 1 - 0.075 = 0.925$$

第五章——搜索策略

一、基本概念（名词解释）

搜索

采用某种策略，在知识库中寻找可利用的知识，从而构建一条代价较小的推理路线，使问题得到解决的过程

盲目搜索

按照**预定的控制策略**进行搜索，在搜索过程中获得的**中间信息不用来改进控制策略**

启发式搜索

在搜索中加入与问题相关的启发性信息，用以指导搜索想着最有希望的方向前进，加速问题的求解过程并找到最优解的过程

启发性信息

可用于指导搜索过程，并且与具体问题相关的信息

二、状态空间

状态空间表示法

很多问题的求解可以用状态空间表示法来表示

状态：状态用以描述问题在求解过程中不同时刻的状态，一般用一个向量表示：

$$S_K = (S_{k0}, S_{k1} \dots)$$

算符：使问题从一个状态转变为另一个状态的操作称为算符。在产生式系统中，**一条产生式规则就是一个算符**。

状态空间：由所有可能出现的状态及一切可用算符所构成的集合称为问题的状态空间。

其中，**状态** S_K 表示：

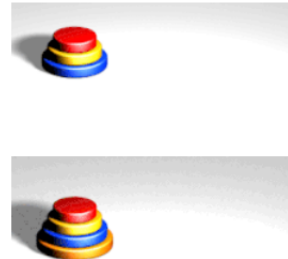
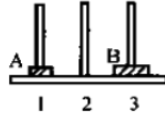
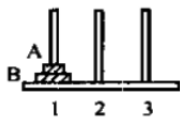
$$S_K = (S_{k0}, S_{k1}, \dots)$$

状态 S_K 表示当前问题处于可能的第 K 个状态， $S_{k0}, S_{k1}, \dots$ 表示本问题在当前的第 k 个状态中，第 $0, 1, \dots$ 个元素的状态（所处位置等）

- 采用状态空间表示方法，首先要把问题的一切状态都表示出来，其次要定义**一组算符**。
- 问题的求解过程是一个不断**把算符作用于状态**的过程。如果使用某个算符后得到的新状态是目标状态，就得到了问题的一个解。这个解就是从初始状态到目标状态所采用**算符的序列**。使用算符最少的解称为**最优解**。

举例：

□ 例5.1 二阶梵塔问题。设有三根钢针，在1号钢针上穿有A、B两个金片，A小于B，A位于B的上面。要求把这两个金片全部移到另一根钢针上；而且规定每次只能移动一片，任何时刻都不能使B位于A的上面。



1、列出所有可能的状态

本问题的状态SK表示为，该二阶梵塔问题有几种可能的，金片A,B的存在状态

Sk0表示在第k个状态下金片A所处位置(1,2,3)，Sk1表示第k个状态下金片B所处位置(1,2,3)，则SK的状态共有下面的：

$$\begin{aligned} S_0 &= (1,1), & S_1 &= (1,2), & S_2 &= (1,3) \\ S_3 &= (2,1), & S_4 &= (2,2), & S_5 &= (2,3) \\ S_6 &= (3,1), & S_7 &= (3,2), & S_8 &= (3,3) \end{aligned}$$

2、找出初始状态集合与目标状态集合

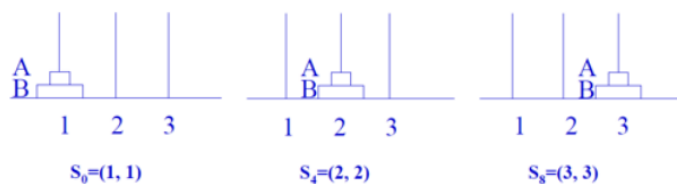
初始时A与B都在1号柱子：

$$S=\{S_0\}$$

目标状态下，两片金片A,B要同时在2号或者3号柱子上：

$$G=\{S_4, S_8\}$$

问题的初始状态集合为 $S = \{S_0\}$ ，目标状态集合为 $G = \{S_4, S_8\}$ 。



3、定义算符

(2) 算符： $A(i,j)$ 及 $B(i,j)$ 。 $A(i,j)$ 表示把A金片从第*i*号钢针移到第*j*号钢针。 $B(i,j)$ 与之同理。算符共有12个：

$A(1,2) A(1,3) A(2,1) A(2,3) A(3,1) A(3,2)$

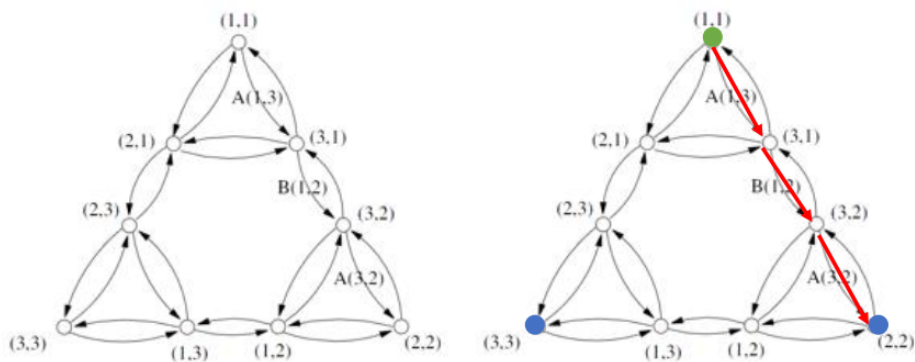
$B(1,2) B(1,3) B(2,1) B(2,3) B(3,1) B(3,2)$

4、根据算符和状态，构造状态空间图

状态 S_n 经过算符A/B后可以得到状态 S_m ，由此得出状态空间图，然后在得到的状态空间图上找出最短路径即可

(3) 在状态空间图中，从初始节点(1,1)到目标节点(2,2)或(3,3)的任何一条通路都是问题的一个解。

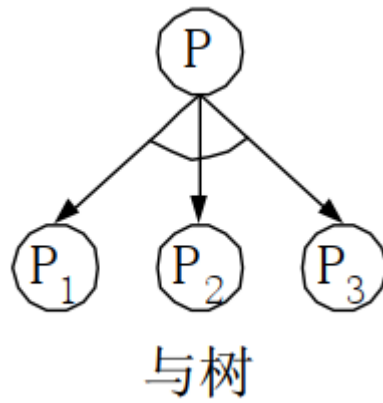
- 其中最短的路径长度是3，它由3个算符组成。
- 例如：从(1,1)开始，通过使用算符 $A(1,3)$, $B(1,2)$, $A(3,2)$ ，可得到(2,2)。



与或树表示法

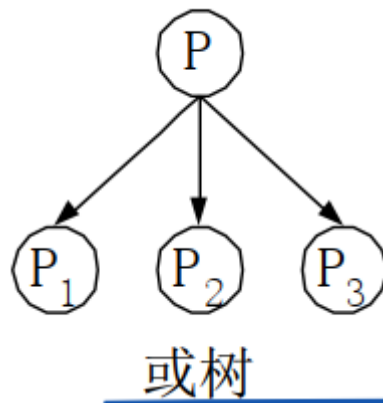
一个复杂的问题可以进行分解，将其分解为几个小问题，根据问题之间的关系有：

与树：（分解得到）



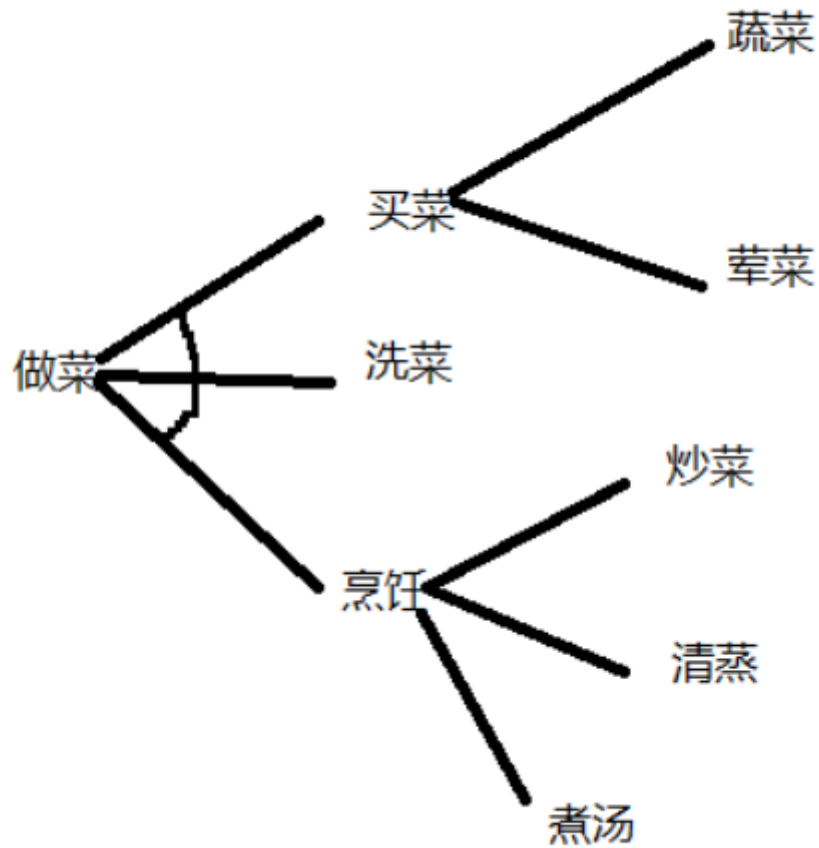
将一个问题P分解为多个小问题，且这些小问题**全部都要解决**

或树：（等价变换得到）



将原问题变为多个容易求解的新问题，且这些新问题**只需要解决一个**就能解决原问题

举个例子：



基本概念

- **本原问题**
 - 不能再分解或变换，而且直接可解的子问题。
- **端节点与终止节点**
 - 在与/或树中，没有子节点的节点统称为端节点；本原问题所对应的节点称为终止节点。
- **可解节点**
 - 在与/或树中，满足下列条件之一者，称为可解节点：
 - 它是一个终止节点；
 - 它是一个“或”节点，且其子节点中至少有一个是可解节点；
 - 它是一个“与”节点，且其子节点全部是可解节点。
- **不可解节点**
 - 关于可解节点三个条件全部不满足的节点。

三、状态空间的搜索策略

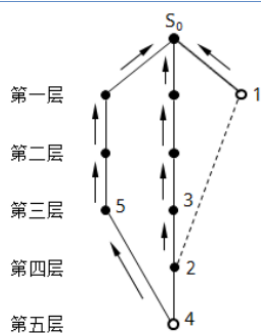
状态空间的一般搜索过程 ==重点==

通过在状态空间中寻找用于解决问题的路径到达目标状态，以解决问题

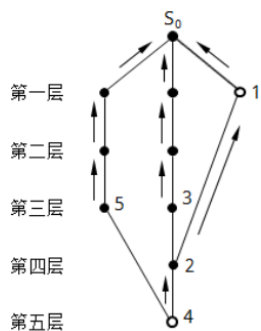
1. 将初始节点 S_0 放入OPEN表中，并建立只包含 S_0 的图
2. 检查OPEN表是否为空，若为空则问题无解，若不为空，往下走
3. 根据特定算法策略，从当前OPEN表中取出首节点（广度优先是先进先出，深度优先是后进先出）放入CLOSED表中
4. 检测是否是目标节点，若是，问题得解，退出；若不是，进行**拓展**：对当前节点应用所有合法算符，将得到的所有邻居状态中**不是当前节点的祖先节点**的节点加入图G中，并记作集合M（暂时先不放入OPEN表，运行下方的逻辑检测，若**这些节点在CLOSED表中出现过，也计入集合M的范围**）
5. 针对拓展得到的子节点集合M中的不同情况，分别进行如下处理：
 - 集合M中**不在OPEN表**中的子节点：设置一个指向当前节点n的指针（将当前指针设置为父结点），并放入OPEN表中
 - 集合M中**已经在OPEN表**中的子节点：需要判断其是否需要修改其指向父结点的指针

♥♥♥重点♥♥♥：如果修改了指针，且路径上有代价，需要更新修改后的子节点的代价 $g(x)$

一个新生成的节点，它可能是第一次被生成的节点，也可能是先前已作为其它节点的子节点被生成过，当前又作为另一个节点的子节点被再次生成。此时，它究竟应选择哪个节点作为父节点？一般由初始节点到该节点的代价来决定，处于代价小的路途上的那个节点就作为该节点的父节点。



- 另外，结点4既是结点2的子结点又是结点5的子结点。当结点2以结点3作为父结点时，由于从 S_0 经中间路径到结点4的代价大于从 S_0 经左边路径到结点4的代价，所以结点4以结点5为父结点。

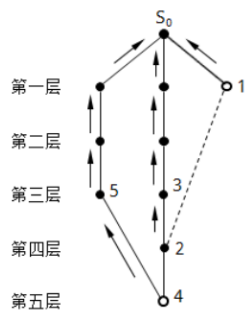


- 但是，经扩展结点1之后，从 S_0 经右边路径到结点4的代价为3，而从 S_0 经左边路径到结点4的代价为4，所以结点4不能再以结点5为父结点，而需要改为以结点2为父结点。此时，搜索图如下图所示。这就是搜索过程步骤6所阐述的内容。

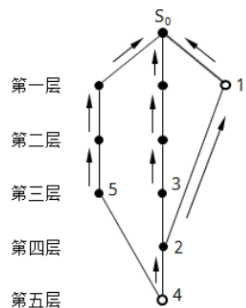
上图表明：子节点4已处于OPEN表中，当子节点1拓展后子节点2的父结点被修改，此时**位于OPEN表中的子节点4的父结点也需要被修改**

- 集合M中**已经在CLOSED表**中的子节点：确定是否需要修改这些节点的后续节点指向其父节点的指针

♥♥重点♥♥：如果修改了指针，且路径上有代价，需要更新修改后的子节点的代价 $g(x)$



- 假设现在要扩展结点1，并且只生成单一的子结点2。
✓ 但是目前结点2已有父结点3，那结点2在先前扩展结点3时已被生成了，现在又作为结点1的子结点被再次生成。此时，为确定哪一个结点作为结点2的父结点，需要计算路径代价。



- 假设每条边的代价为1，则从 S_0 经右边路径到结点2的代价为2，而从 S_0 经中间路径到结点2的代价为4，显然右边路径到结点2的代价较小，因此应修改结点2指向父结点的指针让它指向结点1，即把结点1作为结点2的父结点，不再以结点3作为它的父结点。

上图中，子节点1拓展后的子节点2存在于CLOSED表中，但由于子节点1拓展后从 S_0 到子节点2的路径代价更短，需要将子节点2的父结点重新设置为子节点1

- 按照某种搜索策略对OPEN表中的节点进行重排序
- 跳转回第4步，检查OPEN表是否为空，然后进行循环

搜索图

通过搜索得到的图称为搜索图

搜索树

由搜索图中所有节点及其指向父结点的指针构成的集合是搜索树

盲目搜索

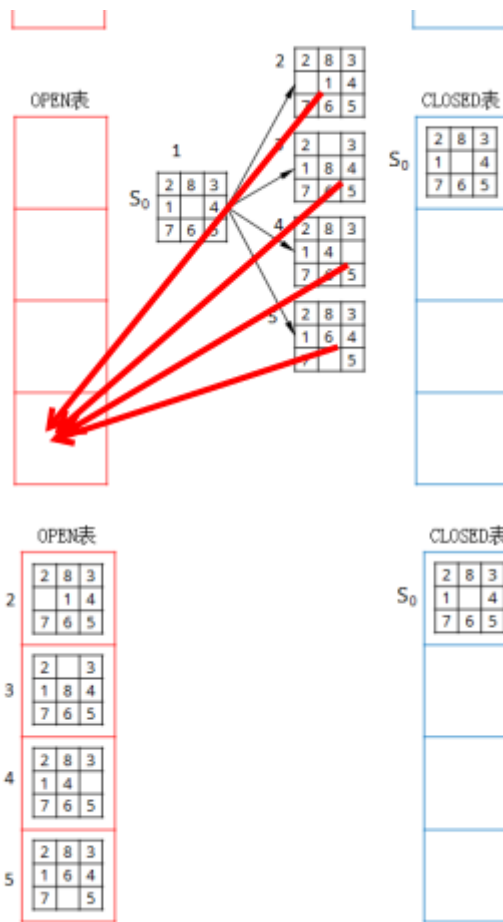
特点：搜索具有盲目性，容易引发**组合爆炸**问题

搜索按照规定的路线进行，不使用与问题相关的**启发性信息**（可用于指导搜索过程且与具体问题相关的信息）

广度优先搜索——OPEN表先进先出

在上述的**状态空间的一般搜索过程**中的拓展子节点的步骤中，将拓展后得到的子节点放入**OPEN表的尾部**，并为每一个子节点配置指向当前节点的父结点指针

拓展时总是按照特定的操作符的顺序进行拓展，将拓展的节点加入尾部，从OPEN表头结点中提取出的首节点就是广度优先的



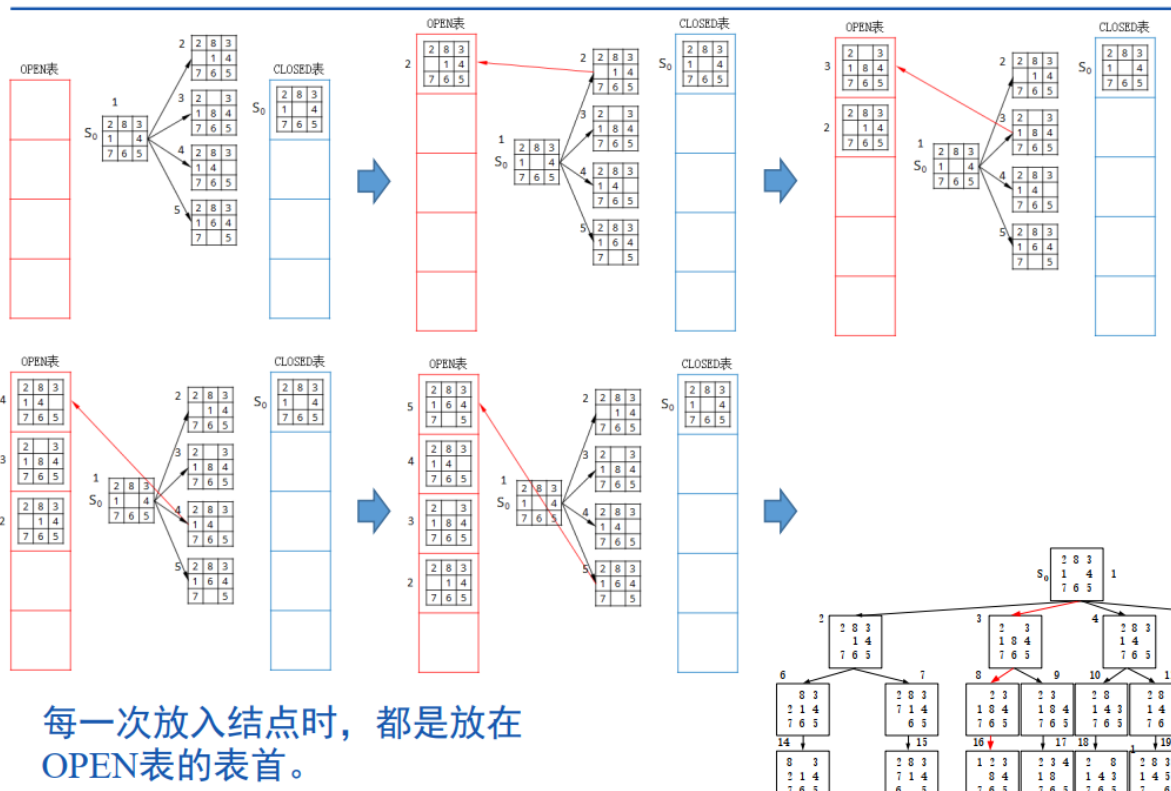
节点拓展后，放入OPEN表时，按照**先后顺序**，**逐个放入OPEN表表尾**，使得当前OPEN表表首是前序未拓展的节点

特点：

- 只要问题有解，广度优先搜索就**一定能得到解**，且解的**路径总是最短**
- 盲目性大，搜索效率低，容易产生组合爆炸

深度优先搜索——OPEN表后进先出

节点拓展后放入OPEN表时，按照**先后顺序**，**逐个放入OPEN表表首**，使得OPEN表表首是**拓展后的最后产生的子节点**，以此类推



保证OPEN表表首节点永远是每次拓展后最后产生的节点

当到达某个子节点，且该子节点既不是目标节点，也不能继续拓展时，才从OPEN表中取出当前节点的兄弟节点进行遍历——即先走到底，然后从底部开始往上找兄弟

特点：

- 路径不一定是最优解
- 深度优先搜索不完备，即使问题有解也可能得不到解

有界深度优先搜索

为深度优先搜索引入搜索深度界限 dm ，当搜索到达深度界限仍未发现目标节点，不再继续拓展，而是选择其兄弟节点

每个节点的深度根据其父结点路径长度，设置为 $d(\text{父结点深度})+1$

特点：

- 有界深度优先搜索可能由于 dm 的限制得不到解
- 即使得到解，其也不一定是最优解

改进方法

1. 合适的选择 dm ，先任意设定一较小的 dm ，若找不到问题的解就逐步增大 dm
2. **最优解**：每次找到目标节点，就将其放入表 R 中，然后让 dm 等于其深度，则后面的每一次解的路径长度都不会超过先前的路径长度，一定能得到最优解

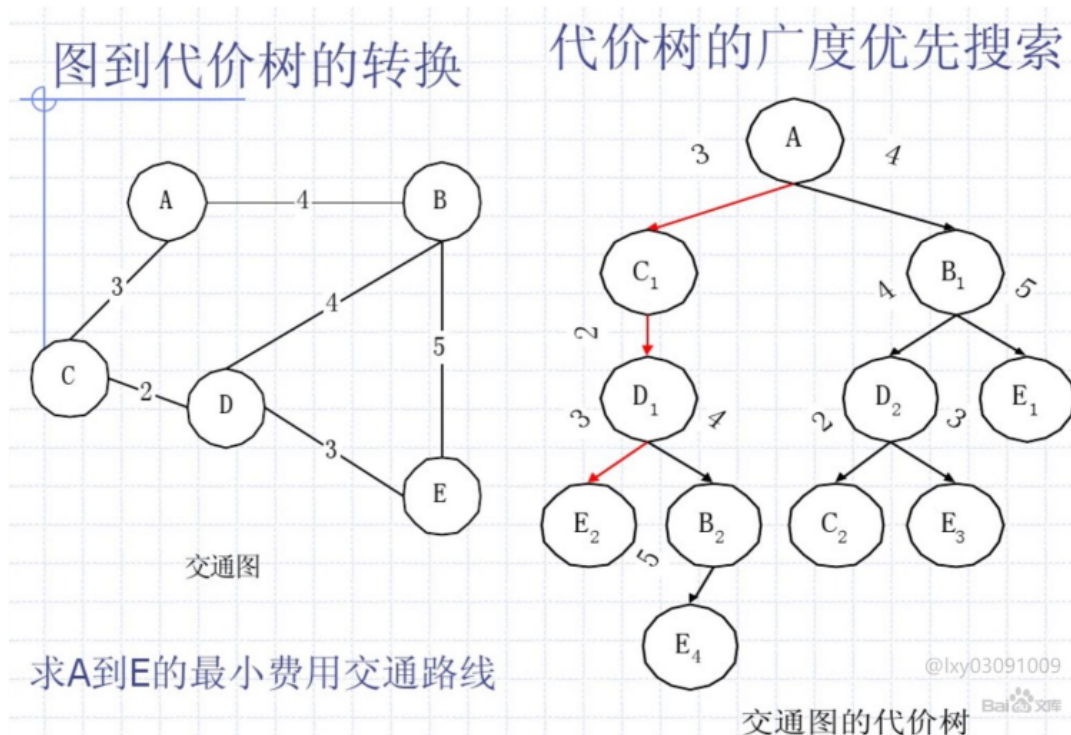
代价树的广度/深度优先搜索

注意：在代价树中，当子节点拓展的子节点集合M需要加入OPEN表时，需要根据父结点指针的变化更新代价

广度优先：

每条路径都标有代价，每次放入OPEN表时，**注意，是全部节点中代价最小**的节点总在最上方，任意时刻OPEN表中的节点都按照代价从小到大排序

- 若问题有解，则代价树一定能求得最优解



深度优先：

从刚拓展出的节点中选择代价最小的节点放入OPEN表首部

- 代价树的深度优先是不完备的，可能得不到解

在代价树的广度优先搜索中，每次都是从OPEN表的全体节点中选择一个代价最小的节点送入CLOSE表进行考察。

在代价树的深度优先搜索中，是从刚扩展出的子节点中选择一个代价最小的节点送入CLOSE表进行考察。

启发式搜索

盲目搜索具有较大的盲目性，会产生大量的无用节点，并引起**组合爆炸**，效率不高

启发式搜索针对问题自身的特性，指导着搜索朝着最有希望的方向前进，针对性强，效率高

启发式搜索要使用与问题有关的，而且能指导搜索过程的启发性信息，并将这些启发性信息用于改正搜索过程，**可以高效求解结构复杂的问题**

估价函数

用于评估节点重要性的函数

$$f(x) = g(x) + h(x)$$

其中 $g(x)$ 表示从初始节点 S_0 到当前节点 x 的**代价**，称其为代价函数

$h(x)$ 表示从当前节点 x 到目标节点 S_g 的最优路径的代价的**估计**，称其为**启发函数**

局部择优搜索

节点拓展后，按照估价函数对每一个拓展后的子节点计算其估价值，然后**选择估价最小的节点**作为下一个考察的节点

若估价函数 $f(x) = g(x)$ ，则局部择优转变为代价树的深度优先

全局择优搜索

每次从OPEN表中选择下一节点时，**从全体OPEN表节点中选择估价值最小的节点**进行考察

若 $f(x) = g(x)$ ，则变为代价树的广度优先搜索

A*算法 ==重点==

- 对OPEN表中的**全体节点**按照估价函数的值从小到大进行排序
- 设 $g(x)$ 是从初始节点 S_0 到当前节点 x 的实际代价， $g^*(x)$ 是从初始节点到当前节点 x 的**最小代价**
- 设启发函数 $h(x)$ ，其代表从当前节点 x 到目标节点 S_g 的最优路径的估计
- 设从当前节点到目标节点的**实际最小代价**为 $h^*(x)$
- 在A算法中，启发函数 $h(x)$ 与实际最小代价 $h^*(x)$ 之间的关系一定为：

$h(x)$ 是 $h^*(x)$ 的下界，即对所有的 x 均有：

$$h(x) \leq h^*(x)$$

- $h(x) = 0$: 由 $g(x)$ 决定节点的优先级, 此时算法就退化成广度优先算法。
- $h(x) \leq h^*(x)$: A*算法保证一定能够找到最短路径。但是当 $h(x)$ 的值越小, 算法将遍历越多的节点, 也就导致算法越慢。
- $h(x) = h^*(x)$: A*算法将找到最佳路径, 并且速度很快。可惜的是, 并非所有场景下都能做到这一点。因为在没有达到终点之前, 很难确切算出距离终点还有多远。
- $h(x) > h^*(x)$: A*算法不能保证找到最短路径, 不过此时会很快。
- $h(x) \gg g(x)$: 此时只有 $h(x)$ 产生效果, 这也就变成了最佳优先搜索。

➤ 由上可知: 通过调节启发函数可以控制算法的速度和精确度。因为在一些情况, 可能未必需要最短路径, 而是希望能够尽快找到一个路径即可。这也是A*算法比较灵活的地方。

启发函数关系	效果
$h(x)=0$	估价函数 $f(x) = g(x)$, 全部由代价考虑, 退化为全局代价树择优搜索
$h(x) \leq h^*(x)$	启发函数被低估, 此时代价函数 $g(x)$ 对估价 $f(x)$ 的影响大, 会优先考虑最短代价的最优路径
$h(x) = h^*(x)$	启发函数=最优路径, 此时A*算法将很快找到最佳路径
$h(x) \geq h^*(x)$	启发函数>最优路径, 被高估, 此时估价函数优先考虑启发函数, 优先选择到目标最近的节点而忽略代价 (路径), 很快但是不一定是最佳路径
$h(x) \gg h^*(x)$	此时估价函数只考虑启发函数, 变为最佳优先搜索

在A*算法中, $g(x)$ 是从初始节点 S_0 到节点 x 的路径的代价, 恒有 $g(x) \geq g^*(x)$ 。而且在算法执行过程中随着更多搜索信息的获得, $g(x)$ 的值呈下降的趋势。

A*算法的性质

最优性: 在 $h(x) \leq h^*(x)$ 的条件下随着启发函数 $h(x)$ 的增大, A*算法的效率越好

单调性: 在A*算法中, 每次拓展一个节点得到的子节点集合M都需要检测其是否存在于OPEN/CLOSED表中, 如果在就要调整其父结点, 这增加了搜索的代价。给启发函数 $h(x)$ 加上**单调性限制**后可以免除上述检查

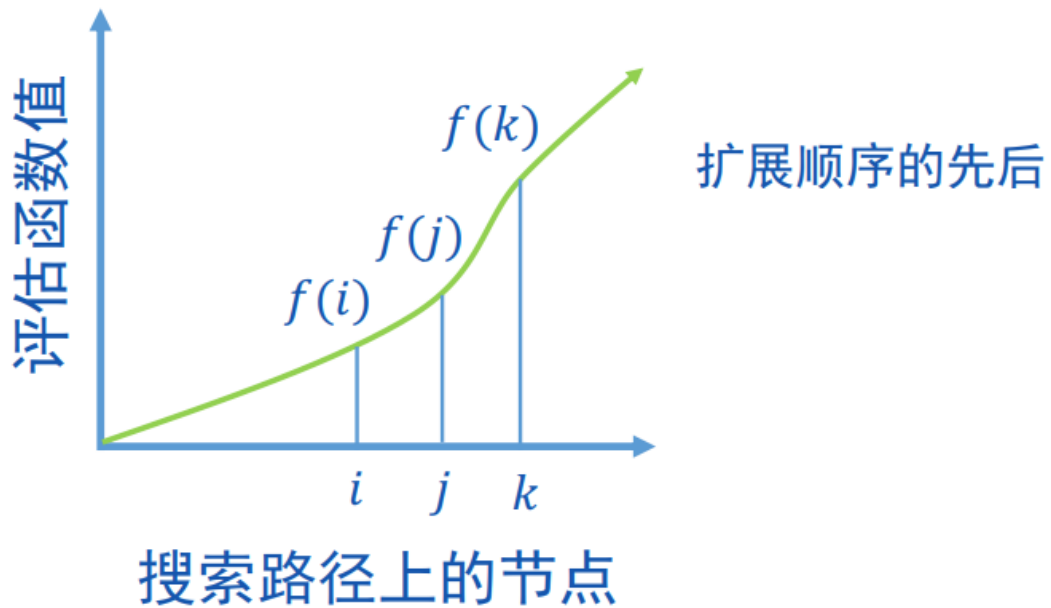
原因：给启发函数加入**单调性限制**后，随着接近目标节点，启发函数应当是**单调递减**的，且递减的幅度是**有限的**

满足：

$$h(\text{当前节点 } x) \leq h(\text{当前节点的子节点 } x_j) + c(\text{当前节点 } x, \text{子节点 } x_j)$$

其中 $c(x, x_j)$ 表示当前节点到子节点的代价

意思是，启发函数递减的同时，相邻节点之间递减的程度不能大于实际代价，进而**保证估值函数递增**



如上图所示，启发函数满足单调限制时，估值函数非递减，搜索时：

- 对于已在CLOSED表中遍历过的节点：已在CLOSED表说明其已被遍历过，其估价函数 $f(x)$ 一定小于当前节点的估价函数，当前路径已经是最优的，跳转回去只会导致估价函数下降，路径变长
- 遍历到的节点的代价永远是最优的： $g(x) = g^*(x)$
- 由于当前节点的代价永远是最小的，那么后续的节点如果与当前节点有关，后续节点到当前节点的代价一定大于当前在OPEN表中的节点的代价

第六章——机器学习

一、基本术语

假设空间

假设

假设是一个机器学习模型对数据的解释或者预测规则

真相

数据集中真实的标签或者结果，是模型学习和预测的目标

假设空间

假设空间是机器学习模型所有可能假设的集合

机器学习过程

在所有假设组成的空间中进行搜索的过程

机器学习目标

找到与训练集所匹配的假设

版本空间

所有与训练数据相一致的假设子集

即所有能正确分类训练样本的假设（模型预测的结果与训练集匹配）

版本空间的收敛

通过版本空间的收敛，模型能找到与数据最符合的假设/假设集合

归纳偏好

机器学习算法在学习过程中对某种类型的假设的偏好

解释

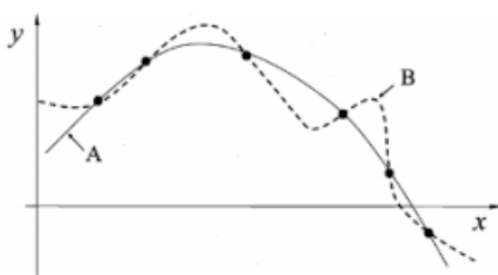
机器学习算法能得到多种假设的曲线，应当选取哪种假设偏好继续训练

奥卡姆剃刀原理

若有多个假设与观察一致，选取最简单的假设

归纳偏好(Inductive Bias):

机器学习算法在学习过程中对某种类型假设的偏好。



存在多条曲线与训练集一致

A还是B?

奥卡姆剃刀原理：若有多个假设与观察一致，则选最简单的那个

在具体的现实问题中，算法的归纳偏好是否与问题本身匹配，大多数时候直接决定了算法能否取得好的性能。

NFL定理

即没有免费午餐定理，一个算法A在某些问题上比另一个算法B好，必存在另一些问题算法B比算法A好

误差

模型的实际预测输出与真实输出之间的差异

经验误差

在**训练集**上的误差，训练集上误差过小可能引起**过拟合**，并不是越小越好

泛化误差

在**新样本**上的误差，泛化误差越小越好，说明在新样本上误差小

模型的评估方法

模型的测试集应当与训练集互斥——留出法

直接将数据集划分为两个互斥的集合——留出法

简单直接，但是不稳定

交叉验证法

将数据集分为k个子集，每次用k-1个训练，剩下的一个子集测试，重复k次取平均值

更稳定但是性能开销大，需要多次计算模型

自助法

有放回的从原始数据集中随机采样生成训练集

验证集

模型评估与选择中用于评估测试的数据集

模型的性能度量

性能度量是衡量模型泛化能力的评价标准，使用不同的性能度量会带来不同的评判标准

查全率与查准率

查准率P：模型预测的所有**正例结果**中预测正确的概率

查准率越高，模型预测的结果的精准度越高

查全率R：模型的所有预测结果中预测正确的概率

查全率越高，模型预测的结果与真实情况越贴近

分类结果混淆矩阵

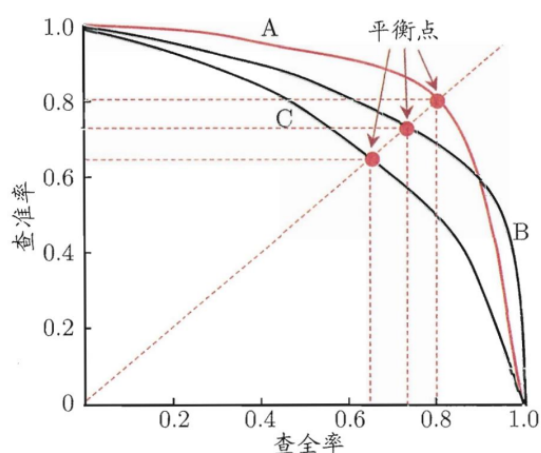
		预测结果	
		正例	反例
真实情况	正例	TP(true positive)	FN(false negative)
	反例	FP(false positive)	TN(true negative)

- 以疾病检测为例：
- 高查准率: 希望所有检测为阳性的病例几乎都是确诊的病人（少错报）。
 - ✓ 可能漏掉一些真正有病的病人（低Recall）。
- 高查全率: 希望几乎所有有病的病人都被检测出来。
 - ✓ 可能会误报一些健康人（低Precision）。

PR图

PR曲线越靠近右上角越好

- 根据学习器的预测结果，按正例可能性大小对样例进行排序，并逐个把样本作为正例进行预测。**PR曲线越靠近右上角越好。**
- 如果一个学习器的PR曲线被另一个学习器完全包住，则可断言后者的性能优于前者。



从左侧PR图可知：

- A 优于 C
- B 优于 C
- A? B

平衡点 (Break-Event Point, BEP)

- P=R 时的取值
- A 优于 B

如何得到一张PR图

通过逐步设置预测结果的阈值进行结果的分类划分，逐样本点计算P和R：

真实标签 预测值

Inst#	Class	Score	Inst#	Class	Score
1	p	.9	11	p	.4
2	p	.8	12	n	.39
3	n	.7	13	p	.38
4	p	.6	14	n	.37
5	p	.55	15	n	.36
6	p	.54	16	n	.35
7	n	.53	17	p	.34
8	n	.52	18	n	.33
9	p	.51	19	p	.30
10	n	.505	20	n	.1

逐个样本设置阈值:

$$\mu = 0.9, P = \frac{1}{1+0} = 1, R = \frac{1}{1+9} = 0.1$$

$$\mu = 0.8, P = \frac{2}{2+0} = 1, R = \frac{2}{2+8} = 0.2$$

$$\mu = 0.7, P = \frac{2}{2+1} = 0.67, R = \frac{2}{2+8} = 0.2$$

$$\mu = 0.6, P = \frac{3}{3+1} = 0.75, R = \frac{3}{3+7} = 0.3$$

...

$$\mu = 0.33, P = \frac{9}{9+9} = 0.5, R = \frac{9}{9+1} = 0.9$$

$$\mu = 0.3, P = \frac{10}{10+9} = 0.53, R = \frac{10}{10+0} = 1$$

$$\mu = 0.1, P = \frac{10}{10+10} = 0.5, R = \frac{10}{10+0} = 1$$

真实情况：正例p、反例n各有10个

$$P = \frac{TP}{TP + FP}$$

$$R = \frac{TP}{TP + FN}$$

15



西安交通大学

XI'AN JIAOTONG UNIVERSITY

模型预测的结果是一个分数形式score，应当按照样本点逐步设置每一个样本点为模型的分类阈值，判断其查全率R和查准率P，得到不同的分类阈值的PR关系，绘制出P-R曲线，再根据trade-off和权衡度量选择不同的P-R关系对应的阈值作为分类依据

二、考试重点--决策树==重点==

什么是决策树

基于“树”结构进行决策分析的模型

每个内部结点对应某种属性的判断，每个叶子结点对应一个决策结果，每个分支对应一种可能的属性结果

如何选择最优划分属性

随着决策树的划分过程的不断进行，分支结点所包含的样本应当尽量属于**同一类别**（例如，某一支节点是大小属性为小，其内部包含的样本应当尽量都是小的样本）

划分方法：

信息增益法

通过**信息熵**判断当前样本集合的“纯度”，信息熵越小，表明当前样本集合混乱度越低，纯度越高，分类效果越好

信息熵:

假定当前样本集合 D 中第 k 类样本所占比例为 $p_k(k = 1, 2, \dots, |y|)$, 则 D 的信息熵定义为:

$$Ent(D) = - \sum_{k=1}^{|y|} p_k \log_2(p_k)$$

信息熵越小表明集合 D 的纯度越高, 当信息熵=0表明样本集合 D 中只有一类

- D 中包含一个类别, 那么 $Ent(D) = 1 \times \log_2 1 = 0$
- D 中包含2个类别:
 - 0.3和0.7, $Ent(D) = -0.3 \times \log_2 0.3 - 0.7 \times \log_2 0.7 = 0.88$;
 - 0.5和0.5, $Ent(D) = -0.5 \times \log_2 0.5 \times 2 = 1$;
 - 0.1和0.9, $Ent(D) = -0.1 \times \log_2 0.1 - 0.9 \times \log_2 0.9 = 0.47$
- D 中包含5个类别:
 - 0.2, $Ent(D) = -0.2 \times \log_2 0.2 \times 5 = 2.32$;
 - 0.05, 0.1, 0.1, 0.05, 0.7, $Ent(D) = 1.46$
- D 中包含10个类别, $Ent(D) = 3.32$

信息增益

假设离散属性 a 有 V 个可能的取值 $\{a^1, a^2, \dots, a^V\}$, D^v 表示样本集中在 a 上取值为 a^v 的样本集合, 可计算出用属性 a 对样本集 D 进行划分所获得的“信息增益”:

$$Gain(D, a) = \overbrace{Ent(D)}^{\text{划分前的信息熵}} - \sum_{v=1}^V \underbrace{\frac{|D^v|}{|D|}}_{\text{分支权重, 即样本数越多, 影响越大}} \overbrace{Ent(D^v)}^{\text{划分后的信息熵}}$$

- 信息增益越大, 则说明使用属性 a 划分所获得的“纯度提升”越大。

信息增益越大, 表明使用属性 a 划分能获得更大的纯度提升效果

即：对于一划分属性：“大小”，有两个可能的取值：“大”、“小”

D^1 表示样本集中属性为大的样本集合

D^2 表示样本集中属性为小的样本集合

计算以属性“大/小”划分的信息增益 $Gain(D, \text{大/小})$ ，首先计算划分前的信息熵 $H(D)$ ，然后计算划分后的信息熵 $H(D^1)$ 和 $H(D^2)$ ，根据上述公式可计算出结果

例子：

<https://www.doubao.com/thread/w7796a96997ec55e6>

ID3算法

以信息增益为准则来选择划分属性的算法，通过每一步求得最大信息增益逐步递归往下进行属性的分类

特点：

- 1、泛化能力差
- 2、训练数据集D需要自带已知目标属性，用于计算初始信息熵
- 3、偏向于存在多种划分属性的样本集合

信息增益对可取值数目较多的属性有所偏好。

- 比如，若将“编号”也作为一个候选划分属性，则根据信息增益度量，将会选择“编号”作为划分属性。
- $Gain(D, a) = Ent(D) - \sum_{v=1}^V \frac{|D^v|}{|D|} Ent(D^v)$
- D 的信息熵是确定的，第二项越小，增益越大。当一个属性在每个样本上的值都不同时，得到的每个分支结点纯度都是最大的（熵最小），因此信息增益最大。
- 这样的决策树显然不具备泛化能力！

引入增益率

Gain_ratio

增益率: $Gain_ratio(D, a) = \frac{Gain(D, a)}{IV(a)}$

其中 $IV(a) = -\sum_{v=1}^V \frac{|D^v|}{|D|} \log_2 \frac{|D^v|}{|D|}$, 称为属性a的**固有值** (intrinsic value)。属性a的可能取值数目越多 (即V越大), $IV(a)$ 的值通常越大。

$$IV(\text{触感}) = -\left(\frac{12}{17} \times \log_2 \frac{12}{17} + \frac{5}{17} \times \log_2 \frac{5}{17}\right) = 0.874$$

$$IV(\text{色泽}) = -\sum_{v=1}^3 \frac{|D^v|}{17} \log_2 \frac{|D^v|}{17} = 1.580$$

$$IV(\text{编号}) = -\sum_{v=1}^{17} \frac{|D^v|}{17} \log_2 \frac{|D^v|}{17} = 4.088$$

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	模糊	凹陷	硬滑	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否

基尼指数

可以用基尼指数来衡量样本集合的纯度:

基尼值: 度量数据集D的纯度

$$Gini(D) = \sum_{k=1}^{|y|} \sum_{k' \neq k} p_k p_{k'} = 1 - \sum_{k=1}^{|y|} p_k^2$$

- 即从数据集D中随机抽取两个样本, 其类别标记不一致的概率
- $Gini(D)$ 越小, 数据集D的纯度越高。
- 有放回摸球, 有不同颜色的球, 其中第k类占比记作 p_k , 那两次摸到的球都是第k类的概率就是 p_k^2 , 那两次摸到的球颜色不一致的概率就是 $1 - \sum p_k^2$, 它的取值越小, 两次摸球颜色不一致的概率就越小, 纯度就越高。

CART决策树使用“基尼指数”来选择划分属性.

CART - Classification and Regression Tree

属性a的基尼指数: $Gini_index(D, a) = \sum_{v=1}^V \frac{|D^v|}{|D|} Gini(D^v)$

➤ 在候选属性集合中，选择使得划分后基尼指数**最小**的属性。

使用基尼指数作为属性划分依据时，选择每次划分后基尼指数最小的属性作为当前阶段的划分依据，基尼指数越小表明当前按照该属性划分后的纯度越高

过拟合与欠拟合 ==重点==

过拟合:

决策树对训练样本过度拟合，背离了真实的样本分布

过拟合会导致模型的泛化能力很差，虽然对训练集样本分类最优，但是对其它数据集分类能力很差

过拟合——决策树模型对训练数据学习的过于“细致”，学习到数据中的噪声和一场，对训练集表现很好，但是对新数据泛化能力差

特征:

- 决策树深度过大，叶节点过多，每个叶节点包含的数据量很少
- 训练集准确率很高，但测试集准确率较低。

欠拟合:

决策树没有充分学习训练数据，模型过于简单，无法捕捉数据的复杂架构

特征：

- 决策树深度过小，分裂次数不足。
- 训练集和测试集的准确率都较低。

如何降低过拟合与欠拟合

测试集法

将数据样本分为训练集和测试集，每次使用训练集训练一层决策树就使用测试集进行性能统计，记录学习曲线，得到最终决策树

阈值法

设定一个阈值作为终止学习的条件，一个结点满足终止条件就停止划分，将其作为叶子节点

阈值——可选信息增益小于某阈值/结点中样本数量比例小于某阈值

剪枝法

剪枝法包括**预剪枝**和**后剪枝**，剪枝过程中需要实时评估剪枝前后决策树的泛化性能是否得到提升，以确定剪枝效果（从而确定最佳过拟合-欠拟合关系）

使用“留出法”——预留一部分数据用作验证集以进行性能评估

1、预剪枝

提前终止某些分支的生长

在每个结点划分前进行性能估计，估计该结点划分**能否提升决策树分类性能**，若**不能提升则停止划分**，并将当前节点标记为叶子节点

性能估计：划分精度——划分正确的概率

- 显著降低过拟合风险，显著减少训练和测试的时间开销。
- 虽然当前划分不能提升性能，但后续划分可能大幅提升性能。
- 预剪枝基于“贪心”本质禁止分支展开，存在欠拟合风险。

预剪枝存在当前划分性能没有提升而后续划分能提升性能的情况，存在欠拟合风险

2、后剪枝

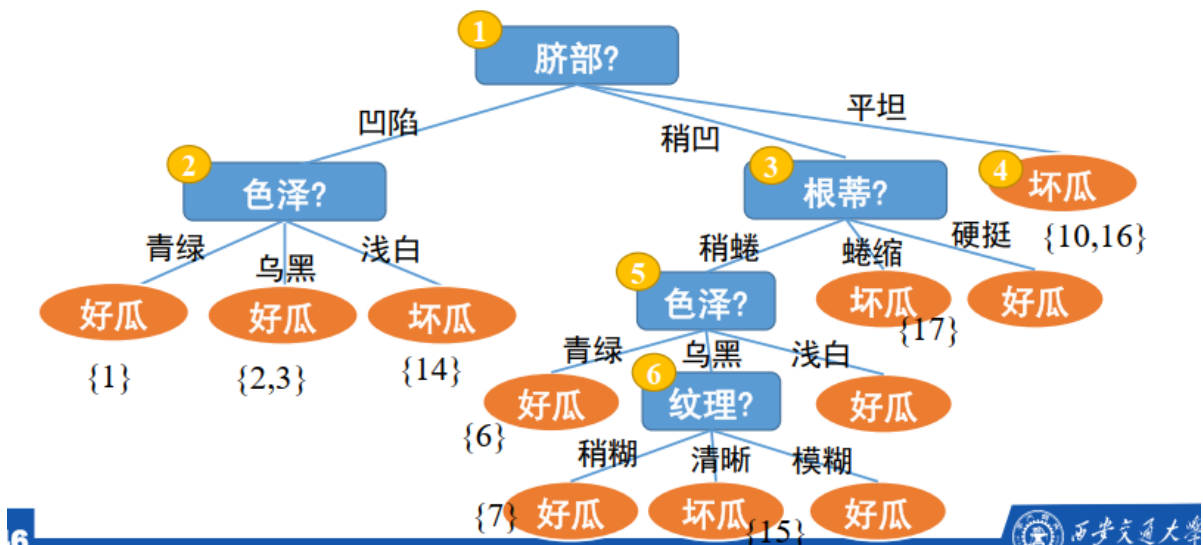
生成一颗完整的树后再回头进行剪枝

step1:删除以此节点为根节点的树

step2:使其成为叶子结点，赋予该节点最常见的分类

step3:对比删除前和删除后的性能是否有所提升，如果有则进行删除，没有则保留

□ **后剪枝**：先从训练集中生成一颗完整的决策树，然后自底向上地对非叶结点进行考察，若将该结点对应的子树替换为叶结点能带来决策树泛化性能的提升，则将该子树替换成叶结点。



特点：

后剪枝的欠拟合风险小，泛化性能优于预剪枝决策树，且相比于预剪枝能保留更多结点，缺点是训练时间更长

连续值与缺失值的处理

连续值——连续属性离散化

例如：使用二分法，将连续的属性“身高”通过二分法等方法选择划分点，将连续的属性“身高”划分为“高的身高”和“矮的身高”两种类别用于排序

例如连续属性“身高”有n个样本点，可以从中任意两个样本点之间计算中位数作为划分点，通过迭代考察这n个划分点划分之后的性能（信息增益或者基尼指数）确定最佳划分点

与离散属性值一样考察这些候选划分点，选取最优的划分点进行样本集合的划分。

$$Gain(D, a) = \max_{t \in T_a} Gain(D, a, t)$$

$Gain(D, a, t)$ 是样本集 D 基于划分点 t 二分后的信息增益

$$= \max_{t \in T_a} Ent(D) - \sum_{\lambda \in \{-, +\}} \frac{|D_t^\lambda|}{|D|} Ent(D_t^\lambda)$$

需注意的是, 与离散属性不同, 若当前结点划分属性为连续属性, 该属性还可作为其后代结点的划分属性。

例如在父结点上使用了“密度 ≤ 0.381 ” 不会禁止在子结点上使用“密度 ≤ 0.294 ”。



缺失值——样本赋权，权重划分

第一步：样本赋权

1、学习开始时，给所有样本赋予权重1

2、通过每种属性中不缺失的样本进行划分属性的选择（计算信息增益/基尼指数）

然后计算下图的三种表达式

给定数据集 D 和属性 a , $\tilde{D}$ 表示 D 中在属性 a 上无缺失值的样本子集

$\tilde{D}^v$ 表示 $\tilde{D}$ 中在属性 a 上取值为 a^v 的样本子集

$\tilde{D}_k$ 表示 $\tilde{D}$ 中属于第 k 类的样本子集，

为每个样本 x 赋予一个权重 w_x ，定义：

$$\rho = \frac{\sum_{x \in \tilde{D}} w_x}{\sum_{x \in D} w_x}, \tilde{p}_k = \frac{\sum_{x \in \tilde{D}_k} w_x}{\sum_{x \in \tilde{D}} w_x} \quad (1 \leq k \leq |y|),$$

$$\tilde{r}_v = \frac{\sum_{x \in \tilde{D}^v} w_x}{\sum_{x \in \tilde{D}} w_x} \quad (1 \leq v \leq V)$$

ρ 无缺失值样本所占比例
 $\tilde{p}_k$ 无缺失值样本中第 k 类所占比例
 $\tilde{r}_v$ 无缺失值样本中在属性 a 上取值 a^v 的样本所占比例

其中， ρ 为属性 a 中无缺失的样本的比例

p_k 为无缺失样本中第 k 类样本所占比例

r_v 为无缺失样本中在属性 a 上取值为 a^v 的比例

对信息增益公式进行推广：

□ 信息增益可推广为：

$$Gain(D, a) = \rho \times Gain(\tilde{D}, a)$$

$$= \rho \times (Ent(\tilde{D}) - \sum_{v=1}^V \tilde{r}_v Ent(\tilde{D}^v))$$

$$Ent(\tilde{D}) = - \sum_{k=1}^{|y|} \tilde{p}_k \log_2 \tilde{p}_k$$

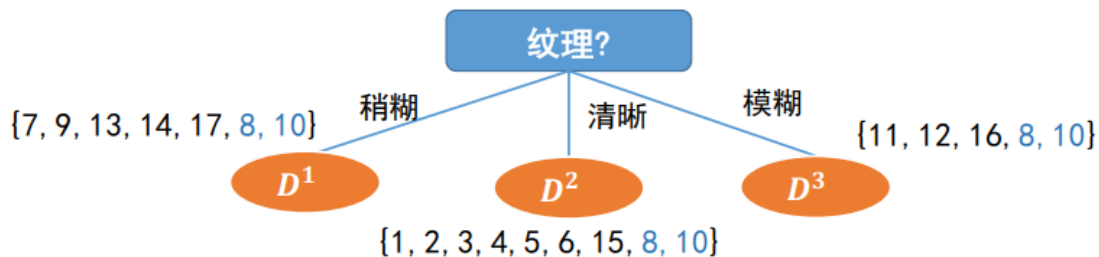
3、进行了划分属性选择后，若当前划分属性中含有缺失值，处理方法如下：

- 若划分属性a上的样本值已知，则权重不变 w_x （若第一次划分，保持为1；若前面经历过缺失值划分，权重不为1）
- 若划分属性a上的样本值**缺失**，则将该缺失的样本编号**同时划分入所有子结点中**，权重调整为：当前子节点所属的类别v的 $rv \times w_x$

rv 即属性a的类别v中无缺失值占总无缺失值的比例

同理，可计算出所有属性在D上的信息增益：

$$\begin{aligned} Gain(D, \text{色泽}) &= 0.252; Gain(D, \text{根蒂}) = 0.171; \\ Gain(D, \text{敲声}) &= 0.145; \boxed{Gain(D, \text{纹理}) = 0.424}; \\ Gain(D, \text{脐部}) &= 0.289; Gain(D, \text{触感}) &= 0.006 \end{aligned}$$



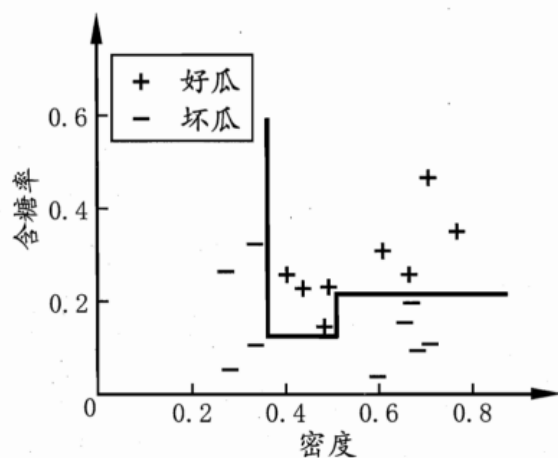
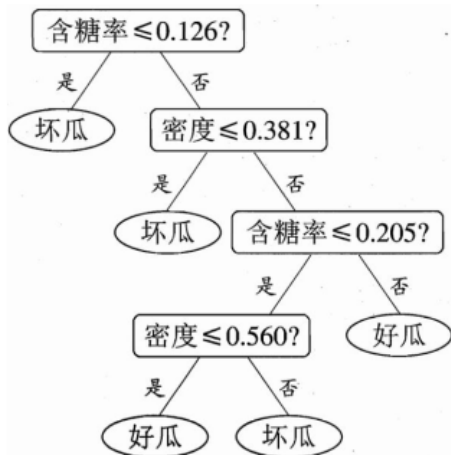
- 编号为8、10的样本在“纹理”上具有缺失值，因此它将同时进入三个分支中，且权重分别调整为5/15、7/15、3/15。
- 其余样本权重保持为1。

例如上图：纹理的无缺失值为15，稍糊中有5个无缺失值，则其 $rv=5/15$ ，划分入该子节点的缺失值权重调整为5/15

多变量决策树

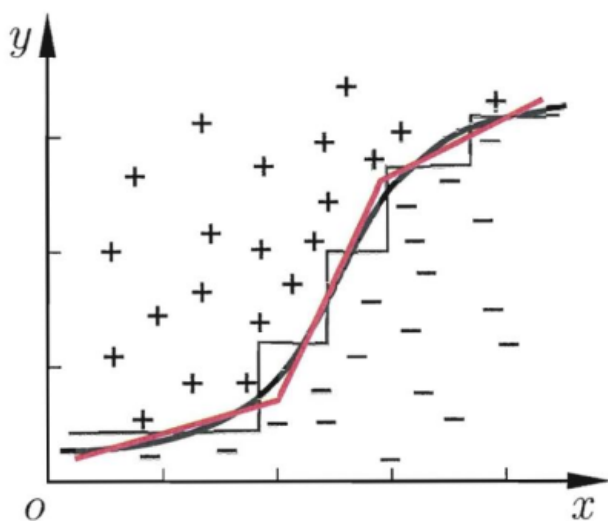
单变量决策树的分类面与轴平行，可解释性好

产生“轴平行”分类面，可解释性好



由上图可知，若使用连续值作为划分属性，可以多次使用候选的划分中心点作为划分属性

但是在学习任务的真实分类边界比较复杂时，需要使用多端划分才能获得较好的近似，此时决策树非常复杂，预测时间开销大

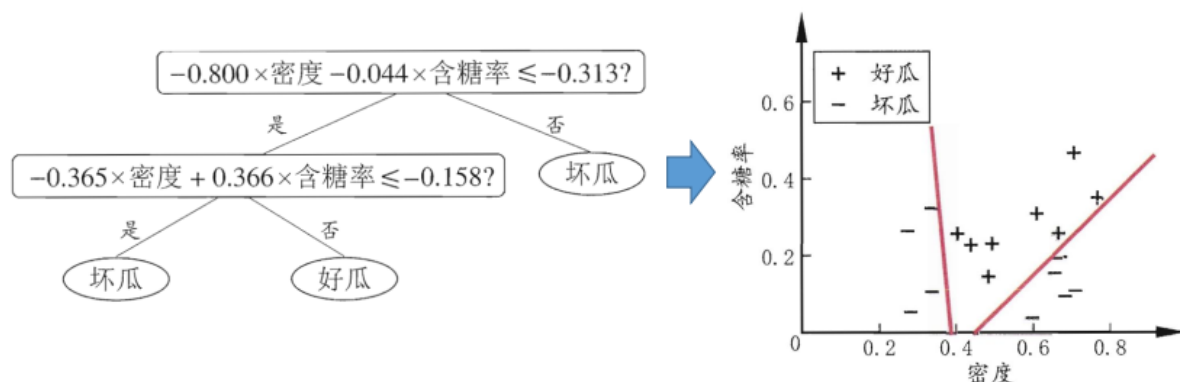


此时决策树非常复杂，预测时间开销很大。

若能适应斜的划分边界，如红色线段所示，模型将大为简化。

多变量决策树：在每个非叶结点不仅考虑一个属性。

“斜决策树”：不是为每个非叶结点寻找最优划分属性，而是建立一个线性分类器

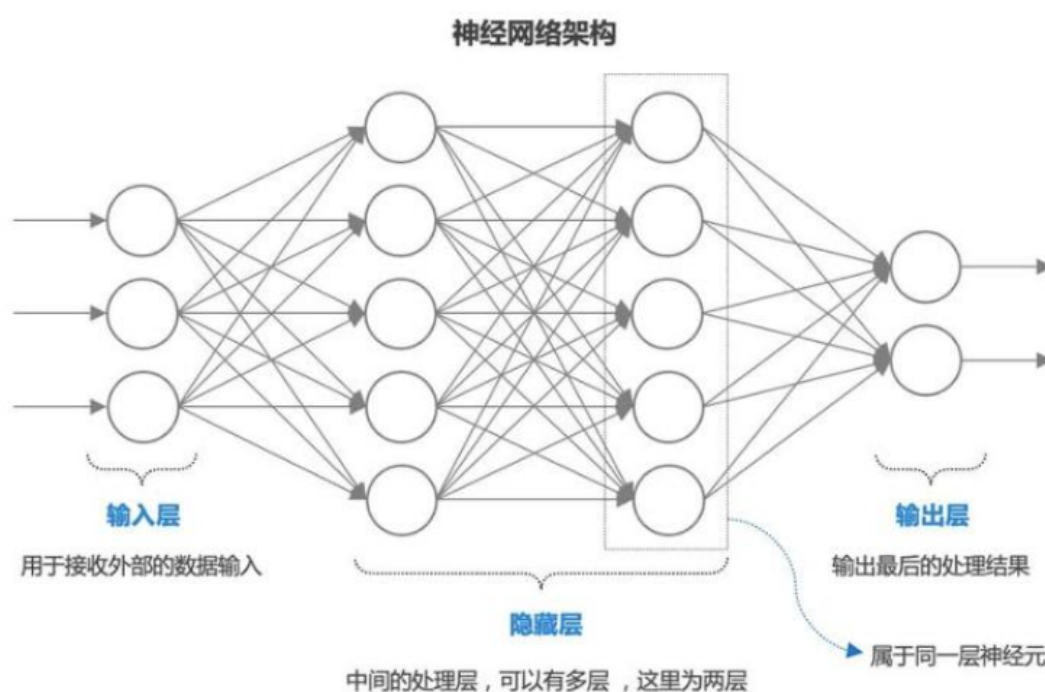


第七章——神经网络

BP算法

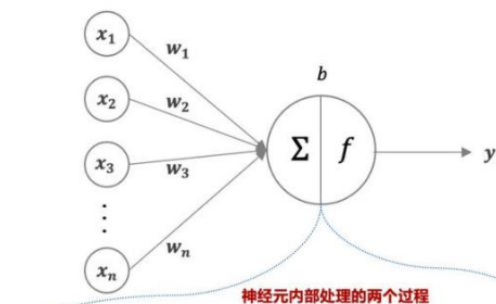
前馈神经网络

神经网络由输入层-隐藏层-输出层组合而成，数据只向前传播而不反向回馈的神经网络叫前馈神经网络



神经元

在单个神经元内部，首先给每个输入变量赋予权重，然后加权求和并加上偏置，最后将加权求和的结果输入到激活函数中决定当前神经元是否启用，最终输出激活函数的结果



采用不同的激活函数会产生不同的结果，
这里以阶跃函数为例，当 $x*w+b \geq 0$ 时，
神经元输出结果为1，否则为0



$$z = x_1 * w_1 + x_2 * w_2 + x_3 * w_3 + \dots + x_n * w_n + b$$

$$y = f(z) = f(x * w + b)$$

$$= \sum_{i=1}^n x_i w_i + b$$

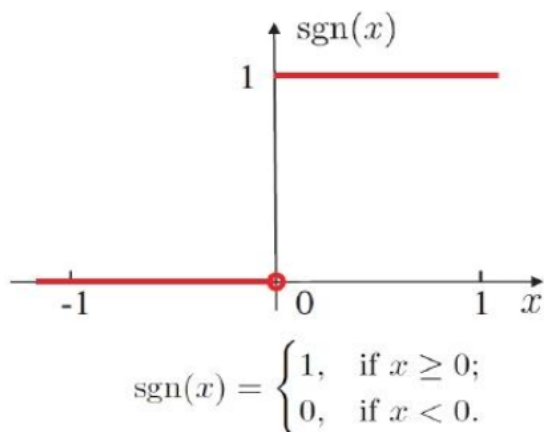
$$= x * w + b$$



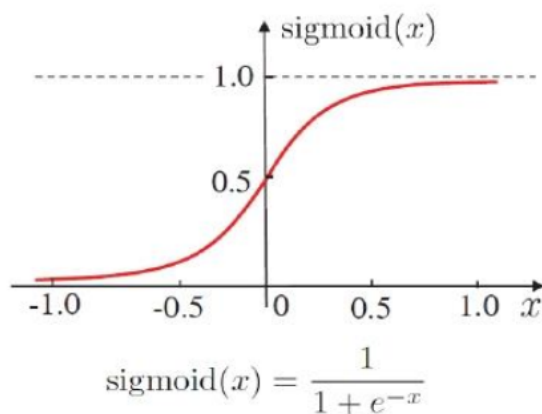
激活函数

激活函数是在人工神经网络的神经元上运行的，负责将非线性特性引入神经网络中的函数

如果不引入激活函数，那么每一层神经元的输出都是上一层的线性组合



(a) 阶跃函数

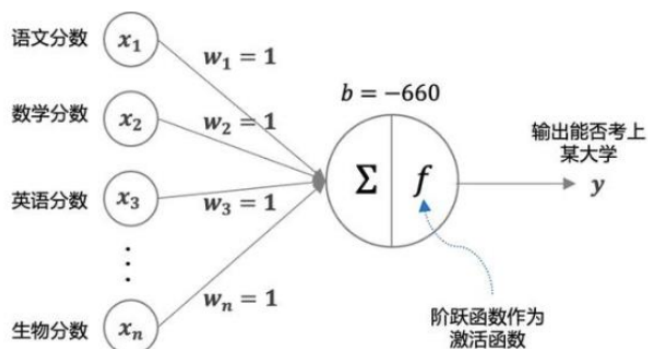


(b) Sigmoid 函数

包括阶跃函数和Sigmoid函数

例如：

想要判断某个同学能否上某个大学的录取分数线，可以用单个神经元来进行决策，该神经元以各学科分数作为输入变量，以该大学以往的录取分数线作为阈值，以阶跃函数作为激活函数，就可以利用该神经元来判断学生能否考上某大学了。



输入 x : 这里输入变量为各个学科的分数值。

权重 w : 高考总分中每一个学科的权重都相同，都为1。

偏置 b : 假设根据以往数据，要考上该大学需要660分，那么偏置 b 设置为-660。

激活函数 f : 采用阶跃函数作为激活函数，当6个学科的总分大于660分时，加权输入大于等于0，那么阶跃函数输出为1，否则为0。

输出 y : 根据激活函数的输出结果，1代表能够考上该大学，0代表不能考上该大学。

前向传播

神经网络的训练过程 == 重点 ==

1、准备数据集，划分为训练集和验证集，将神经网络的参数进行初始化

2、运行前向传播

- 输入数据经过神经网络的每一层，每一层都计算其加权输入和，再加上偏置，然后应用激活函数，最终在输出层生成预测结果（单个神经元的计算步骤）
- 使用一个损失函数衡量预测结果与真实标签之间的差异

3、反向传播

- 计算损失函数关于输出层的梯度，利用链式法则从输出层开始逆向通过网络的每一层，计算损失函数关于每个权重和偏置的梯度

4、参数更新

- 使用梯度下降法等根据计算得到的梯度更新网络的权重和偏置

（梯度代表某个参数对预测结果的影响能力，梯度越高影响能力越大）

新参数 = 旧参数 - 学习率 $\times$ 梯度

- 计算新的参数在训练集或者测试集上的性能

5、重复步骤2-4，直到训练集的性能最优，然后选择在验证集上性能表现最好的参数作为最终的神经网络

BP算法==重点==

损失函数

通过损失函数 $C(w, b)$ 来表示模型拟合的好坏，某一组参数 (w, b) 的损失函数的值与预测值和真实值之间的**差异**有关，一般使用如下损失函数：

用二次损失函数来表示拟合的好坏：

对于每一个样本，拟合误差用如下损失函数表示：

$$\begin{aligned} C(w, b) &= \frac{1}{2} \| y(x) - a \|^2 \\ &= \frac{1}{2} \sum_j (y_j(x) - a_j)^2 \end{aligned}$$

假设有 n 样本数据，则所有样本数据的损失函数表示为：

$$C(w, b) = \frac{1}{2n} \sum_x \| y(x) - a \|^2$$

其中 a_j 和 $y_j(x)$ 表示有 j 个输出，所有预测输出 a_j 与真实标签 $y_i(x)$ 之间的差的平方的求和再除以 2

链式法则

利用损失函数计算梯度时，首先计算损失函数关于输出层的梯度（对输出层求导），然后利用**求导的链式法则**逆向求网络中每一层的权重 w 和偏置 b 关于损失函数之间的梯度

求导的链式法则：

Case 1 $y = g(x) \quad z = h(y)$

$$\Delta x \rightarrow \Delta y \rightarrow \Delta z \qquad \frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$$

Case 2

$$x = g(s) \quad y = h(s) \quad z = k(x, y)$$

$$\frac{dz}{ds} = \frac{\partial z}{\partial x} \frac{dx}{ds} + \frac{\partial z}{\partial y} \frac{dy}{ds}$$

计算实例

- 1、正向传播，算出损失函数
- 2、根据损失函数，逐步从输出层开始计算损失函数 $C(w,b)$ 与每一层的权重 w 之间的梯度关系（偏导数），得到每一层所有权重 w 关于损失函数的梯度

注：此时不更新权重参数 w ，求偏导时的每一层神经元的输出保持不变，权重保持不变

- 3、统一更新所有的权重信息

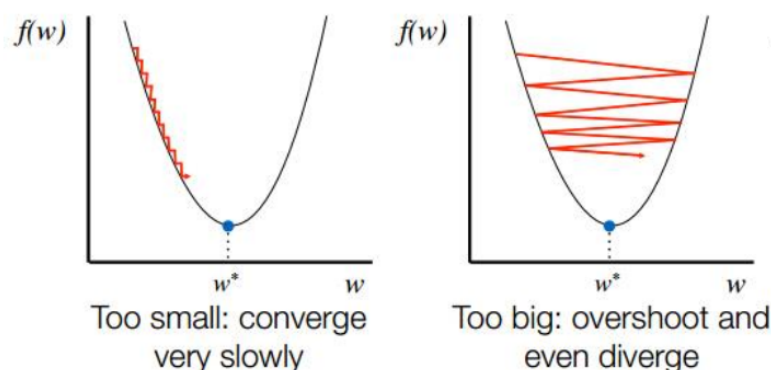
根据**新权重参数=旧参数-学习率 η ×本参数的梯度**计算出新的权重 w'

- 4、再次运行前向传播，计算输出和损失函数

学习速率 η

(1) 学习速率

学习速率 η 直接影响权系数调整时的步长。学习速率过小，导致算法收敛速度缓慢。学习速率过大，导致算法不收敛。学习速率的典型取值 $\eta \approx 0.1$ 。另外学习速率可变。



学习速率过慢会使得算法收敛速度变慢